High-Precision CKKS with Focal Units

Jung Hee Cheon[1,2] and Junho Lee[1]

[1] Seoul National University.
{jhcheon,abab9579}@snu.ac.kr
[2] CryptoLab Inc.

Abstract. In CKKS, higher precision requires a larger scaling factor, increasing modulus consumption during homomorphic arithmetic and leaving less modulus available after bootstrapping under the same modulus budget. We introduce Focused CKKS, which trades unused SIMD slots for precision through focused rescaling and key-switching based on cyclotomic units. These units reduce error in selected focus slots while allowing larger error in the background slots. Using only a $1/k$ fraction of the slots provides approximately k times as many bits of precision at the same scaling factor and CKKS parameter setting. Such a high-precision multiplication costs the same as an ordinary CKKS multiplication, with each focus slot supported by $(k-1)$ background slots. More generally, the construction supports different precision requirements across slots.
We also construct a bootstrapping procedure tailored to Focused CKKS. Only the base modulus at the bottom level needs to grow with the target precision; the remaining bootstrapping computation maintains approximately the same modulus consumption by using fewer slots. By adapting the high-precision bootstrapping based on discrete CKKS and performing its error-cleaning step in Focused CKKS, we further reduce this base modulus by approximately the same factor k.

1 Introduction

Fully homomorphic encryption (FHE) enables computation on encrypted data without revealing the underlying messages. The CKKS scheme [CKKS17] supports approximate arithmetic on real and complex numbers, packing many values into a single ciphertext and processing them in parallel through single-instruction, multiple-data (SIMD) operations. This packing improves throughput by amortizing the cost of homomorphic computation across the slots.

However, some computations require high precision but offer limited SIMD parallelism. For example, computations in private blockchains, scientific computations, and precision-sensitive iterative algorithms may need small numerical errors, and intermediate values may require more precision than the final output. When only a small number of values are processed together, much of the available slot capacity remains unused. Such workloads motivate using this capacity to improve the precision of the values being computed.

Realizing this tradeoff in ordinary CKKS is difficult because precision is controlled by the scaling factor relative to the error magnitude. Increasing the

scaling factor improves precision but increases the modulus consumed by rescaling after multiplication. Under a fixed modulus budget, this reduces the available multiplicative depth and leaves less modulus after bootstrapping. Simply leaving slots unused does not reduce the error in the occupied slots, so higher precision still incurs this modulus cost.

A related tradeoff between packing capacity and plaintext size appears in work on high-precision exact arithmetic, including constructions based on polynomial plaintext moduli [CLPX18,GV25,CHM+25,BFG+26,GZ26] and representations using residues or digit decompositions [BK25,Kim25,CPL26,PZZ26]. Motivated by these tradeoffs, we seek efficient high-precision *approximate* arithmetic: use fewer slots to obtain more bits of precision in each remaining slot, while retaining CKKS's arithmetic operations and modulus consumption per level. We pursue this tradeoff by redistributing error across slots, allowing unused slots to support proportionally higher precision in the slots carrying the message.

1.1 Contributions

Focused CKKS. We construct Focused CKKS, which trades unused slots for precision. Using a $1/k$ fraction of the slots provides approximately k times as many bits of precision at the same scaling factor and CKKS parameter setting. Each focus slot is supported by $(k-1)$ background slots that tolerate larger errors. We give focused encoding, encryption, rescaling, and key-switching procedures, and show that they support homomorphic arithmetic. In particular, a high-precision multiplication costs the same as an ordinary CKKS multiplication, consuming a single level.

Error shaping with focal units. Our construction uses cyclotomic units to reduce error in the focus slots while allowing larger error in the background slots. We use the logarithmic unit lattice to construct units with prescribed error profiles, subject to a balance of error across slots. This method also supports different precision requirements across slots. We also give a subring construction that preserves rotations among the focus slots and a concrete search procedure for finding focal units with low scale overhead.

Focused bootstrapping. We construct focused S2C and C2S transforms and combine them with EvalMod to bootstrap the focus slots. For fixed evaluation depth, focusing reduces the precision-dependent modulus consumption of these stages by a factor of approximately k, apart from ring- and error-dependent overheads. The base modulus at the bottom level still accommodates the full target precision. Thus, under the same modulus budget, the refreshed ciphertext retains more modulus for subsequent computation. We also reduce the remaining precision-dependent modulus consumption by adapting the high-precision bootstrapping based on discrete CKKS [CKSS25], performing its error-cleaning step in Focused CKKS.

1.2 Technical Overview

We work over the cyclotomic field $\mathcal{K} = \mathbb{Q}[X]/(X^N + 1)$ with its ring of integers $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, where N is a power of two.

The desired error profile. A CKKS ciphertext $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ at scale Δ and modulus Q satisfies

$$b + a \cdot \mathsf{sk} = \Delta m + e \pmod{Q},$$

where $\mathsf{sk} \in \mathcal{R}$ is the secret key, $m \in \mathcal{K}$ represents the intended message, and $e \in \mathcal{K}$ is the accumulated error, which carries over all the way to decoding. Write $\sigma_j(e)$ for its value in slot j, and let E be an upper bound on its canonical norm $\|e\|_\infty^{\mathsf{can}} = \max_j |\sigma_j(e)|$. This gives an error bound $|\sigma_j(e)|/\Delta \le E/\Delta$ after decoding. We fix a set J_F of slot indices, called the *focus slots*, and call the remaining slots the *background slots*. Our aim is to improve this bound on the focus slots by maintaining ciphertexts that decrypt to $\Delta m + ue$ with $\|e\|_\infty^{\mathsf{can}} \le E$. Here $u \in \mathcal{R}^\times$ is a unit in the plaintext ring, chosen to have small values $\sigma_j(u)$ on J_F. The decoded error in a focus slot j is then at most $|\sigma_j(u)|E/\Delta$. For a target of p bits of precision, we therefore seek

$$\Delta \gtrsim \|u\|_\infty^{\mathsf{can}}\, E, \qquad \frac{|\sigma_j(u)|E}{\Delta} \lesssim 2^{-p} \quad (j \in J_F).$$

The scale condition controls the quadratic error term in multiplication, as explained below.

Finding a focal unit. Fix the scale Δ, and let $p_0 = \log_2 \Delta - \log_2 E$ be the precision of ordinary CKKS at this scale. The two conditions then read

$$\log_2 \|u\|_\infty^{\mathsf{can}} \le p_0, \qquad \log_2 |\sigma_j(u)| \le p_0 - p \quad (j \in J_F).$$

Since a unit has field norm of absolute value one, $\prod_j |\sigma_j(u)| = 1$, so suppressing error in the focus slots must be balanced by amplification in the background slots. When a $1/k$ fraction of the slots are focus slots, we aim for magnitudes $2^{p_0 - p}$ on the focus slots and 2^{p_0} on the background slots. Their product is one exactly when $\frac{1}{k}(p_0 - p) + \frac{k-1}{k} p_0 = 0$, that is,

$$p = k p_0.$$

Thus using a $1/k$ fraction of the slots yields k times the precision of ordinary CKKS at the same scale. An actual unit need not attain these magnitudes exactly. The logarithmic unit lattice guarantees that we can approximate these magnitudes within a factor depending on the ring.

Two perspectives of encryption. We achieve the focused error profile by going between two perspectives. First is the usual message with shaped error,

$$b + a \cdot \mathsf{sk} = \Delta m + ue \pmod{Q},$$

and the other is a shaped message with usual error,

$$u^{-1}b + u^{-1}a \cdot \mathsf{sk} = u^{-1}\Delta m + e \pmod{Q}.$$

Since u is a unit, multiplying the ciphertext by u or u^{-1} converts between these forms exactly, without introducing error. In particular, we can encode and encrypt $u^{-1}m$ at scale Δ using ordinary CKKS, then multiply the ciphertext by u to obtain the first form.

Key switching and rescaling are the two main operations that introduce fresh error, so their errors must also be focused to stay small on the focus slots. To this end, we apply the ordinary operation to $u^{-1}\mathsf{ct}$, then multiply the result by u. This gives the intended key switch or scale reduction, while the newly introduced error is multiplied by u and the existing error keeps its factor u.

A rotation applies an automorphism κ to the error, giving $\kappa(u)\kappa(e)$, so we require $\kappa(u) = u$ to preserve the factor u. We obtain such rotations by choosing u in a smaller cyclotomic subring $\mathcal{R}' \subset \mathcal{R}$. A small value of u in a single slot of $\mathcal{R}'$ corresponds to several slots of $\mathcal{R}$, which we designate as the focus slots. The automorphisms fixing $\mathcal{R}'$ act as rotations on these slots and leave u unchanged. Together with focused key-switching, they let us rotate the message within the focus slots while preserving the error form.

Multiplication and background slots. Multiplication requires a separate argument. For input ciphertexts ct_i decrypting to $\Delta m_i + ue_i$, multiplication followed by focused relinearization and rescaling gives error $ue_\times$, where

$$e_\times = m_1 e_2 + m_2 e_1 + \frac{u}{\Delta} e_1 e_2 + e_{\mathsf{fresh}}.$$

Here ue_{fresh} is the error introduced by the focused version of relinearization and rescaling. If $\|m_i\|_\infty^{\mathsf{can}} \le 1$, $\|e_i\|_\infty^{\mathsf{can}} \le E$, and $\Delta \ge \|u\|_\infty^{\mathsf{can}} E$, then each of the first three terms is at most E, so $\|e_\times\|_\infty^{\mathsf{can}} \le 3E + \|e_{\mathsf{fresh}}\|_\infty^{\mathsf{can}}$. In particular, when both intended input messages are zero on the background slots, the two cross terms vanish there, stabilizing background slot error with appropriate scale and fresh-error margins.

Focused bootstrapping. CKKS bootstrapping performs encoding conversions $\mathsf{S2C}$ and $\mathsf{C2S}$, homomorphic DFTs between the slot and the coefficient encoding. Focused error does not help here, and in fact it hurts: each coefficient of the error combines its values on all slots, so the focused error ue, of size up to $\|u\|_\infty^{\mathsf{can}} E$ on the background slots, has coefficients inflated by about $\|u\|_\infty^{\mathsf{can}}$. Done naively, $\mathsf{S2C}$ and $\mathsf{C2S}$ must therefore be evaluated at the full precision p, and each of their levels, typically three per transform, consumes about $p + \log_2 E$ bits.

Our approach is to keep as much of S2C and C2S as possible in the focused state. To this end, we first devise focused linear transformations that act only on the focus slots. By the diagonal method [HS14], they consist of focused rotations within the focus slots, plaintext multiplications, and a focused rescale, so that each level consumes only about $p/k + \log_2 E$ bits.

At the coefficient-encoded end of each transform, the error must be ordinary rather than focused. We convert between focused and ordinary error, that is, lose and regain focus, by multiplying by u^{-1} and u. Since our choice of u has a common small value u_F on the focus slots, multiplying a ciphertext supported on the focus slots by u^{-1} turns the focused error ue into ordinary error e, and the scale Δ into $\Delta/u_F \gtrsim 2^p E$. This larger scale accommodates the full precision required by the coefficient encoding, and multiplication by u restores the focus.

Since each level without focus would again cost about $p + \log_2 E$ bits, we stay focused as long as possible, losing focus only at the end of S2C and regaining it at the start of C2S. The main difficulty is that a coefficient-encoded message occupies all slots, while a focused ciphertext holds values only on the focus slots. We resolve this mismatch using the structure of the DFT, as in sparse packing [CHK+18]: the focus slots carry $d = N/(2k)$ values, which we encode as the coefficients of a polynomial in X^k, and the slots of such a polynomial consist of k copies of a d-point DFT. The DFT itself is thus computed on the focus slots. However, the remaining slots cannot simply be filled with rotated copies of its output $\boldsymbol{v}$: the copies occupy contiguous blocks of slots, while the focus slots are every k-th slot, so the shifted focus slots $r + J_F$ must hold $v_r, v_{r+k}, v_{r+2k}, \ldots$ rather than a copy of $\boldsymbol{v}$. We therefore prepare one focused ciphertext per shift, holding these entries, and interleave them when losing focus. Since a fixed permutation of the output can be folded into the DFT at no extra cost, we reorder it so that the entries $v_r, v_{r+k}, \ldots$ for each shift form a block of d/k adjacent entries. Each shift then needs its block repeated k times, which we produce from $k-1$ rotations of the reordered output and a single level of block masks. C2S is handled symmetrically, folding the k copies into one before the inverse DFT.

Regaining focus in C2S would require masking at the large scale Δ/u_F. We avoid this by applying the masks directly to the integer lift of the ciphertext as part of modulus raising, then rounding, so that the masking consumes no modulus.

For EvalMod, a gap of about $p/2$ bits between the base modulus and the scale yields p-bit precision from the standard trigonometric approximation without additional operations, and an inverse-sine correction [LLL+21] can remove this gap at the cost of an additional polynomial evaluation.

Reducing modulus consumption. The bootstrapping above still requires a base modulus of p bits. To remove this requirement, we build on the high-precision bootstrapping based on discrete CKKS [CKSS25], which achieves high precision without repeating a lower-precision bootstrapping as in Meta-BTS [BCC+22]. After modulus raising, it brings the integer overflow into the slots with a low-precision C2S, decomposes it into bits, and cleans the bits to high precision. The

message is then recovered by subtracting the recombined overflow, after S2C, from the raised ciphertext.

We adopt this approach and perform the cleaning in Focused CKKS, reducing its precision-dependent modulus consumption by a factor of about k. Only the focus slot values of the overflow need to be removed, which we arrange by masking the integer lift as above. Since focused cleaning gains precision only on the focus slots, it is performed k times, once for each shift $r + J_F$, to clean all the overflow coefficients. As the cleaning doubles the precision at each step, we change the focal unit along the way, and develop two methods: an iterated cleaning, which strengthens the focal unit at each step, and a tree-like cleaning, which halves the focus slots at each step to perform fewer cleanings at the cost of more modulus. Hence the base modulus no longer needs p bits, while a dependence on $\log p$ remains through the number of cleaning steps.

1.3 Related Work

High-Precision Exact Arithmetic. Several works reduce the cost of high-precision arithmetic by changing the plaintext representation. CLPX [CLPX18] uses polynomial plaintext moduli to support large integers with reduced noise growth. GBFV [GV25] combines this approach with SIMD packing and bootstrapping, while MatriGear [CHM+25] uses a related BFV variant for efficient triple generation over large prime fields. REFHE [BFG+26] supports arithmetic and logical operations on machine words using a tailored number field and encoding. Building on this line, Gao and Zheng [GZ26] introduce a triangle encoding that represents each machine word across multiple CKKS slots and supports efficient refreshing and conversion to Boolean representations.

Another approach decomposes large integers into smaller components. Boneh and Kim [BK25] use nested residue number systems, while Kim [Kim25] and Cha et al. [CPL26] use digit representations with homomorphic carry processing. Peikert et al. [PZZ26] give a general construction for high-precision exact arithmetic over arbitrary integer plaintext moduli, reducing error growth through plaintext decomposition.

Approximate Arithmetic with Polynomial Plaintext Moduli. Polynomial plaintext moduli also support complex-number arithmetic. Bootland et al. [BCIV20] extend CLPX to complex-valued data with plaintext moduli $X^m + b$, and Chen et al. [CIL21] combine this with CKKS-style slot encoding into a somewhat homomorphic scheme with smaller ciphertexts than CKKS when packing few values. In contrast, our work stays within CKKS and supports bootstrapping, trading slot capacity for precision through error shaping with focused rescaling and key switching.

Tuple-CKKS. Cheon et al. [CCKS23] introduce Tuple-CKKS, which decomposes a ciphertext into t ciphertext components and performs multiple-precision arithmetic on them. At high precision, this reduces the modulus consumption

per multiplication by a factor approaching t. The tradeoff is additional computation: tuple multiplication uses $t(t+1)/2$ tensor products and t relinearizations and rescalings [CCKS23, Sec. 5]. It also introduces more involved error growth depending on the tuple length t and the growth of the lower components through successive multiplications. Focused CKKS instead uses background slots within a single ciphertext to support higher precision in the focus slots. It retains one tensor product, one focused relinearization, and one focused rescaling per multiplication, at the cost of reduced slot capacity.

High-Precision CKKS Bootstrapping. High-precision bootstrapping has been pursued through improvements to both modular approximation and the bootstrapping procedure. Lee et al. [LLL+21] combine minimax approximation with an inverse-sine correction, while subsequent work optimizes modular approximation for error variance [LLK+22]. Meta-BTS [BCC+22] increases precision by repeating a lower-precision bootstrapping procedure, trading additional latency for higher precision. EvalRound+ [SSKM25] reduces modulus consumption by allowing lower-precision coefficient-to-slot transforms in one branch of the computation. Choe et al. [CKSS25] combine this approach with integer cleaning to obtain high-precision bootstrapping with improved modulus consumption and latency. Our construction trades slot capacity for precision, using focused transforms and arithmetic to reduce the precision-dependent modulus consumption of bootstrapping.

Concurrent Work. In independent and concurrent work, Dumezy [Dum26] develops a closely related approach to trading unused slots for precision in approximate homomorphic encryption. His construction uses polynomial scaling and shaped rounding, rescaling, and key-switching, with subring units suppressing errors on data slots while allowing larger errors on unused slots. It also includes bootstrapping and conversions to and from CKKS. Dumezy evaluates C2S as a dense, single-level transform using BSGS, with a rescaling cost accommodating the full target precision and shaped rounding restored at its output [Dum26, Secs. 5.1–5.2]. The cost of unfactored transforms becomes significant for larger batches [Dum26, Sec. 7.2]. Our C2S preserves focused error through the inverse DFT, reducing the precision-dependent modulus cost per level by a factor of approximately k. It also admits a multilevel butterfly decomposition with focused rescaling at each level, trading additional levels for fewer rotations and plaintext products.

Last revised: 20 October 2026.

2 Preliminaries

2.1 Notations

Write $[d] = \{0, \ldots, d-1\}$, and for a set $S \subset \mathbb{Z}$ and integers a, b, write $a + bS = \{a + bs : s \in S\}$. Let $\odot$ denote componentwise multiplication, and $\boldsymbol{e}_r$ the r-th

standard basis vector, indexed by $r \in [d]$ in dimension d. For $\boldsymbol{z} \in \mathbb{C}^d$, let ρ denote the cyclic shift $(\rho \boldsymbol{z})_i = z_{(i+1) \bmod d}$. For a permutation π of $[d]$, its permutation matrix $\mathcal{P}_\pi$ satisfies $(\mathcal{P}_\pi \boldsymbol{z})_{\pi(j)} = z_j$. For $d = 2^h$, let $\mathrm{rev} = \mathrm{rev}_d : [d] \to [d]$ denote the bit-reversal which reverses the h-bit binary representation.

Let N be a power-of-two integer, and denote $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, $\mathcal{S} = \mathbb{R}[X]/(X^N + 1)$, $\mathcal{K} = \mathbb{Q}[X]/(X^N + 1)$. We write $[\![x]\!] \in \mathbb{R}^N$ for the coefficient vector of $x \in \mathcal{S}$, and the ℓ^∞, ℓ^2, and ℓ^1-norms on $\mathcal{S}$ are defined with respect to the coefficient vector. For a polynomial $m \in \mathcal{K}$ and modulus Q, $[m]_Q$ denotes the centered remainder mod Q. log denotes the logarithm with base 2.

For a positive integer M, write $\zeta_M = \exp(2\pi \mathsf{i}/M)$, and set $\zeta = \zeta_{2N}$. We index the slots by $j \in [N/2]$, where slot j corresponds to the exponent $5^j \in \mathbb{Z}_{2N}^\times$. For $j \in [N/2]$, $\sigma_j : \mathcal{S} \to \mathbb{C}$ denotes evaluation at ζ^{5^j}, and the canonical embedding is $\sigma(m) = (\sigma_j(m))_{j \in [N/2]} \in \mathbb{C}^{N/2}$. For $m \in \mathcal{S}$, its canonical norm is $\|m\|_\infty^{\mathsf{can}} = \|\sigma(m)\|_\infty$. Write κ_j for the automorphism $X \mapsto X^{5^j}$; then $\sigma(\kappa_j(m)) = \rho^j(\sigma(m))$.

2.2 Error Bounds and Rounding

High-probability bounds. Following the heuristic estimates of CKKS [CKKS17, App. B], we use 6σ as a high-probability bound for a centered, approximately Gaussian random variable with variance σ^2, and $16\sigma_1\sigma_2$ for the product of two independent such variables with variances σ_1^2 and σ_2^2. Throughout, "with high probability" refers to bounds obtained under this convention.

Rounding. The standard coefficient-wise rounding has the following high-probability error bound.

Lemma 1 (Rounding error). *Given a polynomial $m \in \mathcal{K}$, its coefficient-wise rounding $\lfloor m \rceil \in \mathcal{R}$ has error $e_{\mathsf{rnd}} = \lfloor m \rceil - m$ satisfying $[\![e_{\mathsf{rnd}}]\!] \in (-\frac{1}{2}, \frac{1}{2}]^N$. It has a hard error bound*

$$\|e_{\mathsf{rnd}}\|_\infty^{\mathsf{can}} \le N/2.$$

Moreover, if coefficients of $[m]_1 \in \mathcal{K}/\mathcal{R}$ are uniformly and independently distributed, it satisfies a high-probability bound

$$\|e_{\mathsf{rnd}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{rnd}} := 6\sqrt{N/12}.$$

2.3 The CKKS scheme

CKKS [CKKS17] is an RLWE-based HE scheme that supports approximate computation on a vector of real and complex numbers. As an encryption scheme, it consists of the following algorithms, given the error distribution χ.

- $\mathsf{KeyGen}(1^\lambda)$. Given the security parameter λ, samples a secret key $\mathsf{sk} \in \mathcal{R}$ of Hamming weight h, and returns the secret key.
- $\mathsf{Ecd}(m; \Delta)$. Given a message polynomial $m \in \mathcal{K}$ and scaling factor Δ, returns $\lfloor \Delta m \rceil \in \mathcal{R}$.

- $\mathsf{Dcd}(\mathsf{pt}; \Delta)$. Given a plaintext $\mathsf{pt} \in \mathcal{R}$ and scaling factor Δ, returns $\mathsf{pt}/\Delta \in \mathcal{K}$.
- $\mathsf{Enc}_{\mathsf{sk}}(\mathsf{pt})$. Given a plaintext $\mathsf{pt} \in \mathcal{R}$, samples uniform mask $a \in \mathcal{R}_Q$ and error $e_{\mathsf{Enc}} \leftarrow \chi$, and returns $(\mathsf{pt}, 0) + (-a \cdot \mathsf{sk} + e_{\mathsf{Enc}}, a)$.
- $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct})$. Given a ciphertext $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$, returns $[b + a \cdot \mathsf{sk}]_Q$.

On top of the polynomial encoding Ecd, the canonical map induces a vector encoding, called *slot encoding*, given by $\mathsf{Ecd}_{\mathsf{slot}}(\boldsymbol{z}; \Delta) = \mathsf{Ecd}(\sigma^{-1}(\boldsymbol{z}); \Delta)$.

We will use the following terminology.

Definition 1 (Plaintext and Encoding). *We say a plaintext* $\mathsf{pt} \in \mathcal{R}$ *is an encoding of* $m \in \mathcal{K}$ *at a scale* Δ *with error* $e \in \mathcal{K}$*, or simply an encoding of* Δm *with error* e*, if*

$$\mathsf{pt} = \Delta m + e.$$

Definition 2 (Ciphertext and Encryption). *We say a ciphertext* $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ *is an encryption of* $\Delta m \in \mathcal{K}$ *with error* $e \in \mathcal{K}$ *if*

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) = b + a \cdot \mathsf{sk} = \Delta m + e \pmod{Q}.$$

Homomorphic operations. As a homomorphic encryption scheme, the CKKS scheme admits the following operations.

- $\mathsf{SwkGen}(\mathsf{sk}, \mathsf{sk}')$. Given two keys $\mathsf{sk}, \mathsf{sk}' \in \mathcal{R}$, returns a switching key $\mathsf{swk}_{\mathsf{sk} \to \mathsf{sk}'} \in \mathcal{R}_{PQ}^2$ from key sk to sk' for ciphertext modulus Q and auxiliary modulus P, which is an encryption of $P \cdot \mathsf{sk}$ under the key sk'.
- $\mathsf{KS}_{\mathsf{sk} \to \mathsf{sk}'}(\mathsf{ct})$. Given a ciphertext $\mathsf{ct} \in \mathcal{R}_Q^2$ encrypted under key sk, returns a ciphertext $\mathsf{ct}' \in \mathcal{R}_Q^2$ encrypted under key sk', using the switching key $\mathsf{swk}_{\mathsf{sk} \to \mathsf{sk}'} \in \mathcal{R}_{PQ}^2$. Explicitly, it is given as

$$\mathsf{KS}_{\mathsf{sk} \to \mathsf{sk}'}((b, a)) = (b, 0) + \lfloor P^{-1} a \cdot \mathsf{swk}_{\mathsf{sk} \to \mathsf{sk}'} \rceil.$$

- $\mathsf{RS}_q(\mathsf{ct})$. Given a ciphertext $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$, returns a rescaled ciphertext $\mathsf{ct}_{\mathsf{RS}} = (\lfloor q^{-1} b \rceil, \lfloor q^{-1} a \rceil) \in \mathcal{R}_{Q/q}^2$.
- $\mathsf{Add}(\mathsf{ct}_1, \mathsf{ct}_2)$. Given ciphertexts $\mathsf{ct}_1, \mathsf{ct}_2 \in \mathcal{R}_Q^2$, returns the componentwise sum $\mathsf{ct}_1 + \mathsf{ct}_2 \in \mathcal{R}_Q^2$.
- $\mathsf{Mult}(\mathsf{ct}_1, \mathsf{ct}_2)$. Given ciphertexts $\mathsf{ct}_1 = (b_1, a_1), \mathsf{ct}_2 = (b_2, a_2) \in \mathcal{R}_Q^2$ and a relinearization key $\mathsf{swk}_{\mathsf{sk}^2 \to \mathsf{sk}} \in \mathcal{R}_{PQ}^2$, computes $\mathsf{ct}_1 \otimes \mathsf{ct}_2 = (d_0, d_1, d_2) = (b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2)$ and returns

$$\mathsf{ct}_{\mathsf{Mult}} = \mathsf{RS}_{\Delta}((d_0, d_1) + \mathsf{KS}_{\mathsf{sk}^2 \to \mathsf{sk}}((0, d_2))) \in \mathcal{R}_{Q/\Delta}^2.$$

- $\mathsf{Aut}_{\kappa}(\mathsf{ct})$. Given a ciphertext $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ and an automorphism $\kappa \in \mathsf{Gal}(\mathcal{K}/\mathbb{Q})$ with corresponding key $\mathsf{swk}_{\kappa(\mathsf{sk}) \to \mathsf{sk}} \in \mathcal{R}_{PQ}^2$, returns

$$\mathsf{ct}_{\mathsf{Aut}} = \mathsf{KS}_{\kappa(\mathsf{sk}) \to \mathsf{sk}}((\kappa(b), \kappa(a))) \in \mathcal{R}_Q^2.$$

We write $\mathsf{Rot}_j = \mathsf{Aut}_{\kappa_j}$ for homomorphic slot rotation.

Error bounds of homomorphic operations. We list error bounds for some of the above homomorphic operations, fixing the error distribution χ as the coefficient-wise discrete Gaussian distribution of variance σ^2.

Lemma 2 (Encoding error). *Given a message $m \in \mathcal{K}$, $\mathsf{Ecd}(m; \Delta)$ is an encoding of m at scale Δ with error e_{Ecd}. Assuming $[\Delta m]_1$ is distributed uniformly, it satisfies a high-probability bound*

$$\|e_{\mathsf{Ecd}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{rnd}}.$$

Lemma 3 (Encryption error). *Suppose a plaintext $\mathsf{pt} \in \mathcal{R}$ encodes $\Delta m \in \mathcal{K}$ with error e. Then, $\mathsf{Enc}_{\mathsf{sk}}(\mathsf{pt})$ is an encryption of Δm with error $e + e_{\mathsf{Enc}}$, with a high-probability bound*

$$\|e_{\mathsf{Enc}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{Enc}} := 6\sigma\sqrt{N}.$$

Lemma 4 (Rescale error). *Suppose $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ encrypts $\Delta m \in \mathcal{K}$ with error e. Then, the rescaled ciphertext $\mathsf{RS}_q(\mathsf{ct}) = (\lfloor q^{-1}b \rceil, \lfloor q^{-1}a \rceil) \in \mathcal{R}_{Q/q}^2$ encrypts $\frac{\Delta}{q}m$ with error $\frac{e}{q} + e_{\mathsf{RS}}$, with a high-probability bound*

$$\|e_{\mathsf{RS}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{RS}} := 6\sqrt{N/12} + 16\sqrt{Nh/12}.$$

Lemma 5 (Key-Switching error). *Suppose $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ encrypts $\Delta m \in \mathcal{K}$ under secret sk with error e. Then, $\mathsf{KS}_{\mathsf{sk}\to\mathsf{sk}'}(\mathsf{ct})$ encrypts Δm under secret sk' with error $e + e_{\mathsf{KS}}$, with a high-probability bound on e_{KS} as*

$$\|e_{\mathsf{KS}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{KS}} := \frac{Q}{P} \cdot 16\sigma N/\sqrt{12} + E_{\mathsf{RS}}.$$

These bounds follow the noise estimates in CKKS [CKKS17, App. B]; see in particular the proofs of Lemmas 2 and 3 there for rescaling and key switching. The bounds are independent of the size of the message.

2.4 Homomorphic DFT and CKKS Bootstrapping

DFT and coefficient-slot conversion. For a power of two d, define the evaluation matrix

$$\mathcal{V}_d = \left(\zeta_{4d}^{i5^j}\right)_{j,i\in[d]}.$$

It maps coefficient vectors to evaluations at $\zeta_{4d}^{5^j}$, $j \in [d]$. For $m \in \mathcal{S}$, decompose its coefficient vector as $[\![m]\!] = (\boldsymbol{m}_0 \mid \boldsymbol{m}_1)$, where $\boldsymbol{m}_0, \boldsymbol{m}_1 \in \mathbb{R}^{N/2}$. Write $\boldsymbol{m} = \boldsymbol{m}_0 + \mathsf{i} \cdot \boldsymbol{m}_1$ for the paired complex coefficients. Since $(\zeta^{5^j})^{N/2} = \mathsf{i}$, its slot vector is

$$\sigma(m) = \mathcal{V}_{N/2}\boldsymbol{m}.$$

Thus coefficient encoding and slot encoding are related by

$$\mathsf{Ecd}(m; \Delta) = \mathsf{Ecd}_{\mathsf{slot}}(\mathcal{V}_{N/2}\boldsymbol{m}; \Delta).$$

Homomorphic DFT. The operations $\mathsf{S2C}$ and $\mathsf{C2S}$ convert between slot encoding of $\boldsymbol{m}$ and coefficient encoding of m by homomorphically evaluating the following linear maps on the slots:

$$\mathsf{S2C}: \quad \mathsf{Ecd}_{\mathsf{slot}}(\boldsymbol{m}; \Delta) \xrightarrow{\mathcal{V}_{N/2}} \mathsf{Ecd}(m; \Delta),$$
$$\mathsf{C2S}: \quad \mathsf{Ecd}(m; \Delta) \xrightarrow{\mathcal{V}_{N/2}^{-1}} \mathsf{Ecd}_{\mathsf{slot}}(\boldsymbol{m}; \Delta).$$

For $d \geq 2$, the DFT admits a sparse butterfly factorization after bit reversal [CCS19,HHC19]:

$$\mathcal{V}_d \mathcal{P}_{\mathrm{rev}} = \boldsymbol{S}_{d/2} \boldsymbol{S}_{d/4} \cdots \boldsymbol{S}_1, \qquad \boldsymbol{S}_k = I_{d/(2k)} \otimes \begin{bmatrix} I_k & \Omega_k \\ I_k & -\Omega_k \end{bmatrix},$$

where I_k is the identity matrix, $\Omega_k = \mathrm{diag}(\zeta_{8k}^{5^j})_{j \in [k]}$, and $\otimes$ denotes the Kronecker product. Each factor has at most three nonzero cyclic diagonals. The bit reversal is absorbed into the coefficient ordering, and the inverse transform uses the inverse factors.

To evaluate these factors homomorphically, we use the diagonal method [HS14, Sec. 4.3]. For $A \in \mathbb{C}^{d \times d}$, it writes

$$A\boldsymbol{z} = \sum_{t \in [d]} \boldsymbol{a}_t \odot \rho^t(\boldsymbol{z}), \qquad \boldsymbol{a}_t = (A_{i,(i+t) \bmod d})_{i \in [d]},$$

using plaintext multiplications and rotations, followed by one rescale. For $d = bg$, baby-step/giant-step (BSGS) evaluation reduces the rotation count to $b + g - 2$ [HS18, Sec. 6]. For $1 \leq \ell \leq \log d$, grouping the butterfly factors into ℓ balanced consecutive groups gives $O(d^{1/\ell})$ nonzero cyclic diagonals per group. Evaluating each group with BSGS and one rescale uses $O(\ell d^{1/(2\ell)})$ rotations and ℓ levels in total [CCS19,HHC19]. This trades modulus consumption against rotation and key-switching costs.

CKKS bootstrapping. Bootstrapping restores the ciphertext modulus consumed by homomorphic operations. A contemporary CKKS bootstrapping algorithm [CHK+18,CCS19] consists of the following steps:

- $\mathsf{S2C}$ applies a homomorphic DFT to convert an encryption of $\mathsf{Ecd}_{\mathsf{slot}}(\boldsymbol{m})$ into an encryption of its coefficient encoding m.
- $\mathsf{ModRaise}$ lifts the ciphertext (b, a) from the base modulus q to $Q > q$, giving $b + a \cdot \mathsf{sk} = m + qI \pmod{Q}$ for an integer polynomial $I \in \mathcal{R}$.
- $\mathsf{C2S}$ applies a homomorphic inverse DFT to obtain an encryption of $\mathsf{Ecd}_{\mathsf{slot}}(\boldsymbol{m} + q\boldsymbol{I})$, where $\boldsymbol{I}$ is the paired complex coefficient vector of I.
- $\mathsf{EvalMod}$ approximately removes $q\boldsymbol{I}$ by modular reduction on the real and imaginary parts, returning an encryption of $\mathsf{Ecd}_{\mathsf{slot}}(\boldsymbol{m})$ at a refreshed modulus q' with $q < q' < Q$.

2.5 Cleaning

Cleaning reduces the error of messages lying near a discrete set [DMPS24], for instance by $h_1(x) = 3x^2 - 2x^3$ on $\{0, 1\}$ [CKK20]. We consider a general cleaning polynomial.

Definition 3 (Cleaning polynomial [AKP24, Sec. 3.4]). *Let $\mathcal{D} \subset \mathbb{R}$ be a finite set and $M \geq 1$. A polynomial $P \in \mathbb{R}[X]$ is an order-M cleaning polynomial for $\mathcal{D}$ if it is an order-M Hermite interpolation of the identity on $\mathcal{D}$, i.e.,*

$$P(b) = b, \qquad P^{(i)}(b) = 0 \quad (1 \leq i \leq M), \qquad b \in \mathcal{D}.$$

By Taylor's theorem with the integral remainder [AKP24, Thm. 4], for $\epsilon > 0$ we have

$$|P(b + \delta) - b| \leq K_{P,\epsilon} |\delta|^{M+1} \qquad \text{for all } b \in \mathcal{D},\ \delta \in \mathbb{C},\ |\delta| \leq \epsilon,$$

where $K_{P,\epsilon} = \max\{|P^{(M+1)}(t)|/(M+1)! : t \in \mathbb{C},\ |t - b| \leq \epsilon,\ b \in \mathcal{D}\}$. For example, h_1 is the order-1 cleaning polynomial for $\{0, 1\}$ of minimal degree [CKK20,DMPS24], and satisfies $|h_1(b + \delta) - b| \leq 3|\delta|^2 + 2|\delta|^3$ [BKSS24, Lem. 1]; this holds for $\delta \in \mathbb{C}$ as well, since $h_1(b + \delta) - b = \pm 3\delta^2 - 2\delta^3$.

2.6 The Logarithmic Unit Lattice

We recall the logarithmic unit lattice and cyclotomic units following Cramer et al. [CDPR16]. The logarithmic embedding is given by

$$\mathrm{Log} : \mathcal{K}^\times \to \mathbb{R}^{N/2}, \qquad a \mapsto \big(\log|\sigma_j(a)|\big)_{j \in [N/2]}. \tag{1}$$

Since a unit $u \in \mathcal{R}^\times$ has field norm of absolute value 1, and the embeddings occur in conjugate pairs,

$$\sum_{j \in [N/2]} \log|\sigma_j(u)| = \frac{1}{2} \log|\mathrm{Norm}_{\mathcal{K}/\mathbb{Q}}(u)| = 0.$$

By Dirichlet's unit theorem, $\mathrm{Log}\, \mathcal{R}^\times$ is a full-rank lattice in

$$V_0 = \left\{ x \in \mathbb{R}^{N/2} : \sum_{j \in [N/2]} x_j = 0 \right\}.$$

Cyclotomic units. Identifying X with ζ, put

$$b_j = \frac{\zeta^{5^j} - 1}{\zeta - 1} \in \mathcal{R}^\times \quad (j \in [N/2] \setminus \{0\}), \qquad C = \langle \zeta, b_j : j \in [N/2] \setminus \{0\} \rangle.$$

The cyclotomic-unit group C is generated multiplicatively by the b_j and the roots of unity, whose logarithmic embeddings vanish. For our power-of-two cyclotomic ring, the vectors $\{\mathrm{Log}(b_j) : j \in [N/2] \setminus \{0\}\}$ form a $\mathbb{Z}$-basis of $\mathrm{Log}\, C$, a full-rank sublattice of V_0 [CDPR16, Lem. 2.7 and Thm. 2.8]. Here Cramer et al. [CDPR16] index the embeddings by $\mathbb{Z}_{2N}^\times / \{\pm 1\}$, which we identify with $[N/2]$ via $j \mapsto 5^j$.

Covering radius. For a full-rank lattice $L \subset V_0$, define its ℓ^∞ covering radius by

$$\gamma(L) = \sup_{t \in V_0} \min_{v \in L} \|t - v\|_\infty.$$

Lemma 6 (Round-off). *If $\{\mathbf{b}_j\}_j$ is a basis of L and $t = \sum_j c_j \mathbf{b}_j$, then $v = \sum_j \lfloor c_j \rceil \mathbf{b}_j$ satisfies*

$$\|t - v\|_\infty \le \frac{1}{2} \sum_j \|\mathbf{b}_j\|_\infty.$$

In particular, $\gamma(L)$ is finite.

Proof. Apply the triangle inequality to $t - v = \sum_j (c_j - \lfloor c_j \rceil)\mathbf{b}_j$, using $|c_j - \lfloor c_j \rceil| \le 1/2$.

3 Focal Unit and Rounding

We condition the rounding by a fixed cyclotomic unit $u \in \mathcal{R}^\times$, which we call a 'focal unit', redistributing the rounding error across slots.

3.1 Focused rounding and its error

We define the focused rounding as follows.

Definition 4 (Focused rounding). *For a unit $u \in \mathcal{R}^\times$ and a polynomial $m \in \mathcal{K}$, we define*

$$\lfloor m \rceil_u = u \cdot \lfloor u^{-1} m \rceil \in \mathcal{R}. \tag{2}$$

Taking $u = 1$ recovers the ordinary rounding.

Since $u^{-1} \in \mathcal{R}$, focused rounding satisfies

$$\lfloor a + qI \rceil_u = \lfloor a \rceil_u + qI \qquad (a \in \mathcal{K},\ I \in \mathcal{R},\ q \in \mathbb{Z}_{>0}).$$

Consequently, for positive integers $q \mid Q$, $\lfloor a/q \rceil_u \bmod Q/q$ is well-defined for $a \in \mathcal{R}_Q$.

As we show below, it has the error scaled by the unit u.

Proposition 1 (Focused rounding error). *Given a polynomial $m \in \mathcal{K}$, the focused rounding $\lfloor m \rceil_u$ has error $u \cdot e_{\mathsf{rnd}}$ with a hard bound*

$$\|e_{\mathsf{rnd}}\|_\infty^{\mathsf{can}} \le N/2.$$

Proof. Observe that

$$\lfloor m \rceil_u - m = u \cdot (\lfloor u^{-1} m \rceil - u^{-1} m) = u \cdot e_{\mathsf{rnd}},$$

where $e_{\mathsf{rnd}} = \lfloor u^{-1} m \rceil - u^{-1} m$. Lemma 1 on $u^{-1} m$ gives the bound.

In particular, at i-th slot, the error has magnitude $|\sigma_i(u)| \cdot |\sigma_i(e_{\mathsf{rnd}})|$. We call $|\sigma_i(u)|$ the *error factor* at the slot. Slots of error factor below 1 receive less error than they would under ordinary rounding, and slots above 1 receive more.

The following proposition formalizes the balance of the gain and loss in error magnitude.

Proposition 2 (Conservation of error factors). *For every unit $u \in \mathcal{R}^\times$, the product of error factors is one:*

$$\prod_i |\sigma_i(u)| = 1, \qquad \textit{equivalently} \qquad \sum_i \log_2 |\sigma_i(u)| = 0. \tag{3}$$

Proof. $\prod_{i\in[N/2]} |\sigma_i(u)|^2 = |\mathrm{Norm}_{\mathcal{K}/\mathbb{Q}}(u)| = 1$.

Hence, we focus on minimizing errors in some slots, paying for larger error in the other slots.

3.2 Choice of the unit

Suppose we want δ_i bits of extra precision at slot i relative to the ordinary rounding. This can be achieved by taking the error factor of $|\sigma_i(u)| = 2^{-\delta_i}$, and Proposition 2 yields a fundamental restriction $\sum_i \delta_i = 0$.

Instead of finding the unit exactly, we find an approximation through the log-unit lattice $\mathrm{Log}\,\mathcal{R}^\times \subset V_0$, where $V_0 = \{x \in \mathbb{R}^{N/2} : \sum_{j\in[N/2]} x_j = 0\}$, and its concrete sublattice $\mathrm{Log}\,C$ from Section 2.6.

Theorem 1 (Existence of close unit). *Let $\gamma = \gamma(\mathrm{Log}\,C)$. For every precision profile $\delta \in V_0$, there exists a close unit $u \in C$ satisfying*

$$\|\mathrm{Log}(u) - (-\delta)\|_\infty \le \gamma, \quad \textit{equivalently} \quad 2^{-\delta_i-\gamma} \le |\sigma_i(u)| \le 2^{-\delta_i+\gamma}. \tag{4}$$

Proof. Since $\mathrm{Log}\,C$ is a full-rank sublattice of V_0, the conclusion follows from the definition of the covering radius, which is finite by Lemma 6.

Here, we chose $\mathrm{Log}\,C$ with basis given by cyclotomic units, since the log unit lattice $\mathrm{Log}\,\mathcal{R}^\times$ does not have a known closed-form basis. By Theorem 1, one can find the unit with desired error factor with a deviation of up to γ bits.

In a power-of-two cyclotomic ring, every unit u can be multiplied by a power of X to obtain a real unit, fixed by complex conjugation [Lem12, Sec. 2, Examples 1–2]. Since $|\sigma_i(X)| = 1$ at every slot, this changes neither $\mathrm{Log}(u)$ nor $\|u\|_\infty^{\mathsf{can}}$, and preserves all bounds on the embedding magnitudes. We therefore choose such real units when constructing focal units.

The focal unit. We specifically consider focusing the precision to a certain set of slots, while the other slots are left unused.

Fix a target precision $p > 0$, and let $\varnothing \neq J_F \subsetneq [N/2]$ be the set of slots to focus on.

Definition 5 (Focal unit). *A cyclotomic unit $u \in C$ is a p-bit* focal unit *for focus slots J_F if the error factors of the focus slots are smaller by at least p bits, i.e.,*

$$|\sigma_j(u)| \le 2^{-p} \cdot \|u\|_\infty^{\mathsf{can}}, \qquad j \in J_F.$$

Proposition 3 (Construction of focal unit). *Let $\gamma = \gamma(\operatorname{Log} C)$. For every $p > 0$, there is a p-bit focal unit $u \in C$ for focus slots J_F such that, with $\hat{p} = p + 2\gamma$,*

$$\frac{2|J_F|}{N}\hat{p} - \gamma \le \log \|u\|_\infty^{\mathsf{can}} \le \frac{2|J_F|}{N}\hat{p} + \gamma,$$

and

$$\log|\sigma_j(u)| \le -\left(1 - \frac{2|J_F|}{N}\right)\hat{p} + \gamma \quad (j \in J_F).$$

Proof. Set target as

$$\delta_j = \begin{cases} \left(1 - \frac{2|J_F|}{N}\right)\hat{p}, & j \in J_F, \\ -\frac{2|J_F|}{N}\hat{p}, & j \in [N/2] \setminus J_F. \end{cases}$$

Since $\sum_j \delta_j = 0$, Theorem 1 gives a unit $u \in C$ with $|\log|\sigma_j(u)| + \delta_j| \le \gamma$ for every $j \in [N/2]$. This unit satisfies the desired conditions.

While p determines the precision of focus slots, $\|u\|_\infty^{\mathsf{can}}$ will decide the required size of the scale factor. Hence, the goal of the search algorithm is to find the smallest $\|u\|_\infty^{\mathsf{can}}$ given the precision p.

Subring-based construction. While we could search across all the units of $\mathcal{R}$, it is easier to search in a subring $\mathcal{R}' \subset \mathcal{R}$. Furthermore, this choice grants the rotation between focus slots via the automorphisms fixing $\mathcal{R}'$.

Fix power-of-two $k \ge 2$ with $2k \mid N$. Let $\mathcal{R}' = \mathbb{Z}[Y]/(Y^{2k} + 1)$ be a subring of $\mathcal{R}$ by $Y = X^{N/(2k)}$, with slot projections $\sigma'_i(m) = m(\zeta_{4k}^{5^i})$ for slots $i \in [k]$. Consider a unit $u' \in (\mathcal{R}')^\times$ which lifts to $u \in \mathcal{R}$. Since 5 has order k modulo $4k$, every slot $j \in [N/2]$ satisfies

$$\sigma_j(u) = u'\big(\zeta^{5^j N/(2k)}\big) = u'\big(\zeta_{4k}^{5^j}\big) = \sigma'_{j \bmod k}(u').$$

Hence, the error factor of u at slot j is the same as that of u' at slot $j \bmod k$.

Now we fix $J'_F = \{0\} \subset [k]$, so that its lift consists of

$$J_F = k\big[\tfrac{N}{2k}\big] \subset [N/2].$$

Let $C' \subset (\mathcal{R}')^\times$ be the cyclotomic units of $\mathcal{R}'$, generated by ζ_{4k} and $b'_i = (\zeta_{4k}^{5^i} - 1)/(\zeta_{4k} - 1)$ for $i \in [k] \setminus \{0\}$, and let $\gamma' = \gamma(\operatorname{Log} C')$. By Proposition 3 applied to $\mathcal{R}'$, we have a p-bit focal unit $u' \in C'$ for J'_F. Choose u' real, as above, and replace it by $-u'$ if necessary so that $u_F := \sigma'_0(u') > 0$. Then, $\sigma_j(u) = u_F$

for every $j \in J_F$. Also, the bounds hold for u verbatim, so u is a p-bit focal unit with

$$\frac{1}{k}\hat{p} - \gamma' \le \log \|u\|_\infty^{\mathsf{can}} \le \frac{1}{k}\hat{p} + \gamma',$$

where $\hat{p} = p + 2\gamma'$. Moreover, the set of focus slots J_F is closed under automorphisms fixing $\mathcal{R}'$, and such automorphisms act as rotations on the focus slots.

In practice, we search the logarithmic cyclotomic-unit lattice directly. Let B have columns $\mathrm{Log}(b_i')$ for $i \in [k] \setminus \{0\}$. For integer exponents $\boldsymbol{x}$, put $u' = \prod_i (b_i')^{x_i}$ and $\boldsymbol{v} = B\boldsymbol{x} = \mathrm{Log}(u')$. We solve the mixed-integer linear program

$$\min_{\boldsymbol{x} \in \mathbb{Z}^{k-1},\, c \in \mathbb{R}} c \quad \text{subject to} \quad v_0 \le -\Big(1 - \frac{1}{k}\Big)p, \qquad v_i \le c \quad (i \in [k] \setminus \{0\}).$$

Since $\sum_i v_i = 0$, the largest background coordinate is at least p/k, so the achieved gain $\max_i v_i - v_0$ is at least p. The objective minimizes the overhead subject to this focus-depth constraint. From a feasible integer solution, we reconstruct u' and its inverse and check their product using exact integer arithmetic, then reevaluate the embeddings in multiprecision. Table 1 reports the resulting gain and overhead $\max_i v_i = \log \|u'\|_\infty^{\mathsf{can}}$.

Table 1. Focal units found by integer programming. We report the achieved precision gain / overhead $\log \|u\|_\infty^{\mathsf{can}}$, in bits.

p	$k = 2$	$k = 4$	$k = 8$	$k = 16$
16	17.80 / 8.90	20.08 / 5.09	17.79 / 3.09	18.28 / 2.74
32	33.06 / 16.53	32.83 / 8.55	33.29 / 5.25	34.80 / 3.90
64	66.12 / 33.06	65.67 / 17.10	65.65 / 9.09	72.76 / 5.69
128	129.70 / 64.85	129.41 / 32.97	131.20 / 17.32	131.24 / 9.88
256	256.85 / 128.43	258.45 / 64.89	258.90 / 33.46	258.21 / 17.52

4 Focused CKKS

A focal unit shapes both rounding error and errors introduced by homomorphic operations. We present a modified homomorphic encryption scheme parameterized by a fixed unit $u \in \mathcal{R}^\times$, where the homomorphic operation error is of the form $u \cdot e$ with e following the standard error distributions.

Following Definition 1 and Definition 2, we say that $\mathsf{pt} \in \mathcal{R}$ encodes $\Delta m \in \mathcal{K}$ with error $u \cdot e \in \mathcal{K}$ if

$$\mathsf{pt} = \Delta m + u \cdot e,$$

and $\mathsf{ct} \in \mathcal{R}_Q^2$ encrypts $\Delta m \in \mathcal{K}$ under secret sk with error $u \cdot e \in \mathcal{K}$ if

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) = \Delta m + u \cdot e \pmod{Q}.$$

4.1 Encoding and Encryption

Definition 6 (Focused encoding). *Given a message $m \in \mathcal{K}$, its u-focused encoding is*

$$\mathsf{Ecd}^{[u]}(m; \Delta) = \lfloor \Delta m \rceil_u .$$

Proposition 4 (Focused encoding error). *The u-focused encoding of $m \in \mathcal{K}$ encodes m at scale Δ with error $u \cdot e_{\mathsf{Ecd}}$. Assuming $[u^{-1} \cdot \Delta m]_1$ is uniformly distributed, it satisfies a high-probability bound*

$$\|e_{\mathsf{Ecd}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{rnd}}.$$

Proof. Set $e_{\mathsf{Ecd}} = \lfloor u^{-1}\Delta m \rceil - u^{-1}\Delta m$. Then

$$\mathsf{Ecd}^{[u]}(m; \Delta) = \Delta m + u e_{\mathsf{Ecd}}.$$

Under the stated uniformity assumption, Lemma 1 gives $\|e_{\mathsf{Ecd}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{rnd}}$ with high probability.

Definition 7 (Focused encryption). *Given a plaintext $\mathsf{pt} \in \mathcal{R}$, u-*focused encryption *is*

$$\mathsf{Enc}_{\mathsf{sk}}^{[u]}(\mathsf{pt}) = u \cdot \mathsf{Enc}_{\mathsf{sk}}(u^{-1} \cdot \mathsf{pt}) = (\mathsf{pt}, 0) + u \cdot (-a \cdot \mathsf{sk} + e_{\mathsf{Enc}}, a) \in \mathcal{R}_Q^2,$$

where e_{Enc} is sampled from the error distribution χ, and $a \in \mathcal{R}_Q$ is sampled uniformly.

Security is unaffected since u is invertible, and $\mathsf{Enc}_{\mathsf{sk}}(u^{-1} \cdot \mathsf{pt})$ is an ordinary RLWE sample.

Proposition 5 (Focused encryption error). *Suppose $\mathsf{pt} \in \mathcal{R}$ encodes Δm with error ue. Then, $\mathsf{Enc}_{\mathsf{sk}}^{[u]}(\mathsf{pt})$ encrypts Δm with error $u \cdot (e + e_{\mathsf{Enc}})$, with high-probability bound*

$$\|e_{\mathsf{Enc}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{Enc}}.$$

Proof. Writing $(b', a') = \mathsf{Enc}_{\mathsf{sk}}^{[u]}(\mathsf{pt})$, we have

$$b' + a' \cdot \mathsf{sk} = \mathsf{pt} + u e_{\mathsf{Enc}} = \Delta m + u(e + e_{\mathsf{Enc}}) \pmod{Q}.$$

The bound on e_{Enc} is the ordinary encryption-error bound of Lemma 3.

Proposition 6 (Decryption correctness and precision). *Suppose a ciphertext $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ encrypts $\Delta m \in \mathcal{K}$ with error ue, with message and error bound $\|m\|_\infty^{\mathsf{can}} \le 1$, $\|e\|_\infty^{\mathsf{can}} \le E$. The ciphertext decrypts correctly to $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) = \Delta m + ue$ if*

$$\Delta + \|u\|_\infty^{\mathsf{can}} E < Q/2.$$

Furthermore, at slot $j \in [N/2]$, $m' = \mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}); \Delta)$ has slot error of

$$|\sigma_j(m') - \sigma_j(m)| \le \frac{|\sigma_j(u)| \cdot E}{\Delta}.$$

Proof. Put $v = \Delta m + ue$. Since $\|[\![v]\!]\|_\infty \le \|v\|_\infty^{\mathsf{can}} \le \Delta + \|u\|_\infty^{\mathsf{can}} E < Q/2$, centered reduction modulo Q recovers v. Decoding gives $m' = m + ue/\Delta$, hence

$$|\sigma_j(m' - m)| = \frac{|\sigma_j(u)||\sigma_j(e)|}{\Delta} \le \frac{|\sigma_j(u)|E}{\Delta}.$$

Corollary 1 (Precision of the focus slots). *Suppose $u \in \mathcal{R}$ is a p-bit focal unit for slots $J_F \subset [N/2]$, and the ciphertext ct satisfies the hypotheses and modulus condition of Proposition 6. Suppose the scaling factor Δ is set to be*

$$\Delta \ge \|u\|_\infty^{\mathsf{can}} E.$$

Then, focus slots $j \in J_F$ have at least p bits of absolute precision, i.e.,

$$|\sigma_j(m') - \sigma_j(m)| \le 2^{-p}.$$

In particular, if $|J_F| = N/(2k)$ with $k \ge 2$, then for fixed choice of ring and error bound $E > 0$, we can choose u to attain p bits of precision with scaling factor $\log \Delta = p/k + O(1)$.

Proof. For $j \in J_F$, the definition of a focal unit and Proposition 6 give

$$|\sigma_j(m' - m)| \le \frac{|\sigma_j(u)|E}{\Delta} \le 2^{-p} \frac{\|u\|_\infty^{\mathsf{can}} E}{\Delta} \le 2^{-p}.$$

For the final claim, Proposition 3 supplies a unit with $\log \|u\|_\infty^{\mathsf{can}} = p/k + O(1)$. Taking $\Delta = \lceil \|u\|_\infty^{\mathsf{can}} E \rceil$ gives the stated scale bound; the hidden constant depends on the ring and E.

Remark 1. As we will show later, the choice $\Delta \ge \|u\|_\infty^{\mathsf{can}} E$ is taken to prevent growth of the background slots. Specifically, it shows that the scale factor has a lower bound set by the error bound $E = \Omega(\sqrt{Nh})$, independent of the precision requirement.

4.2 Rescale and Key-Switching

The focused rescale is defined through focused rounding, while the focused key-switching is obtained by key-switching $u^{-1} \cdot \mathsf{ct}$.

Definition 8 (Focused rescale). *Given a ciphertext $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ and modulus $q \mid Q$, its* u-focused rescale *is*

$$\mathsf{RS}_q^{[u]}(\mathsf{ct}) = u \cdot \mathsf{RS}_q(u^{-1} \cdot \mathsf{ct}) = \left(\lfloor b/q \rceil_u \,,\ \lfloor a/q \rceil_u \right) \in \mathcal{R}_{Q/q}^2.$$

Proposition 7 (Focused rescale error). *Suppose $\mathsf{ct} \in \mathcal{R}_Q^2$ encrypts Δm with error ue. Then, $\mathsf{RS}_q^{[u]}(\mathsf{ct}) \in \mathcal{R}_{Q/q}^2$ encrypts $\frac{\Delta}{q} m$ with error $u \cdot (\frac{e}{q} + e_{\mathsf{RS}})$, with high-probability bound*

$$\|e_{\mathsf{RS}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{RS}}.$$

Proof. The ciphertext $u^{-1} \cdot \mathsf{ct}$ encrypts $\Delta u^{-1} m$ with error e. By Lemma 4, ordinary rescaling gives decryption

$$\frac{\Delta}{q} u^{-1} m + \frac{e}{q} + e_{\mathsf{RS}} \pmod{Q/q}, \qquad \|e_{\mathsf{RS}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{RS}}.$$

Multiplying by u gives the claim.

Definition 9 (Focused key-switching). *Given a ciphertext* $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ *and a key-switching key* $\mathsf{swk}_{\mathsf{sk}\to\mathsf{sk}'} \in \mathcal{R}_{PQ}^2$, u-focused key switching *is*

$$\begin{aligned} \mathsf{KS}^{[u]}_{\mathsf{sk}\to\mathsf{sk}'}(\mathsf{ct}) &= u \cdot \mathsf{KS}_{\mathsf{sk}\to\mathsf{sk}'}(u^{-1} \cdot \mathsf{ct}) \\ &= (b, 0) + u \cdot \lfloor \frac{1}{P} (u^{-1} a) \cdot \mathsf{swk}_{\mathsf{sk}\to\mathsf{sk}'} \rceil. \end{aligned}$$

Proposition 8 (Focused key-switching error). *Suppose* $\mathsf{ct} \in \mathcal{R}_Q^2$ *is an encryption of* Δm *under secret key* sk *with error* ue. *Then,* $\mathsf{KS}^{[u]}_{\mathsf{sk}\to\mathsf{sk}'}(\mathsf{ct})$ *is an encryption of* Δm *under secret key* sk' *with error* $u \cdot (e + e_{\mathsf{KS}})$, *with high-probability bound*

$$\|e_{\mathsf{KS}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{KS}}.$$

Proof. The ciphertext $u^{-1} \cdot \mathsf{ct}$ encrypts $\Delta u^{-1} m$ with error e under sk. By Lemma 5, ordinary key switching gives decryption under sk' of

$$\Delta u^{-1} m + e + e_{\mathsf{KS}} \pmod{Q}, \qquad \|e_{\mathsf{KS}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{KS}}.$$

Multiplying by u gives the claim.

4.3 Addition, Multiplication and Automorphism

Homomorphic addition remains the same. Plaintext-ciphertext multiplication uses focused rescaling, while ciphertext-ciphertext multiplication also uses focused relinearization. A general automorphism changes the unit; for automorphisms fixing u, the homomorphic automorphism is defined analogously using focused key-switching.

Proposition 9 (Addition with focused error). *Suppose* $\mathsf{ct}, \mathsf{ct}' \in \mathcal{R}_Q^2$ *are encryptions of* Δm, $\Delta m'$ *with error* ue, ue', *respectively. Then,* $\mathsf{ct} + \mathsf{ct}'$ *is an encryption of* $\Delta(m + m')$ *with error* $u \cdot (e + e')$.

Furthermore, if $\|e\|_\infty^{\mathsf{can}} \le E$ *and* $\|e'\|_\infty^{\mathsf{can}} \le E'$,

$$\|e + e'\|_\infty^{\mathsf{can}} \le E + E'.$$

Proof. Adding the decryption identities gives $\Delta(m + m') + u(e + e')$ modulo Q. The error bound follows from the triangle inequality.

Proposition 10 (Plaintext-ciphertext multiplication with focused error). *Suppose* $\mathsf{pt} \in \mathcal{R}$ *encodes* Δm_1 *with error* ue_1*, and* $\mathsf{ct} \in \mathcal{R}_Q^2$ *encrypts* Δm_2 *with error* ue_2*. For* $\Delta \mid Q$*, the ciphertext* $\mathsf{RS}_{\Delta}^{[u]}(\mathsf{pt} \cdot \mathsf{ct})$ *encrypts* $\Delta m_1 m_2$ *with error* $ue_\times$*, where*

$$e_\times = m_1 e_2 + m_2 e_1 + \frac{u}{\Delta} e_1 e_2 + e_{\mathsf{RS}}.$$

If $\|m_1\|_\infty^{\mathsf{can}}, \|m_2\|_\infty^{\mathsf{can}} \le 1$ *and* $\|e_1\|_\infty^{\mathsf{can}}, \|e_2\|_\infty^{\mathsf{can}} \le E$*, then*

$$\|e_\times\|_\infty^{\mathsf{can}} \le 2E + \frac{\|u\|_\infty^{\mathsf{can}}}{\Delta} E^2 + E_{\mathsf{RS}}.$$

Proof. Before rescaling, the decryption is congruent modulo Q to

$$(\Delta m_1 + ue_1)(\Delta m_2 + ue_2) = \Delta^2 m_1 m_2 + u\big(\Delta(m_1 e_2 + m_2 e_1) + ue_1 e_2\big).$$

Apply Proposition 7 and the canonical norm inequalities to get the result.

Proposition 11 (Ciphertext-ciphertext multiplication with focused error). *Suppose* $\mathsf{ct}_1, \mathsf{ct}_2 \in \mathcal{R}_Q^2$ *are encryptions of* Δm_1*,* Δm_2 *with error* ue_1*,* ue_2*, respectively. For* $\Delta \mid Q$*, define the ciphertext multiplication as*

$$\mathsf{Mult}^{[u]}(\mathsf{ct}_1, \mathsf{ct}_2) := \mathsf{RS}_{\Delta}^{[u]}(\mathsf{Relin}^{[u]}(\mathsf{ct}_1 \otimes \mathsf{ct}_2))$$

where $\mathsf{Relin}^{[u]}((d_0, d_1, d_2)) = \mathsf{KS}_{\mathsf{sk}^2 \to \mathsf{sk}}^{[u]}((0, d_2)) + (d_0, d_1)$*.*

Then, $\mathsf{Mult}^{[u]}(\mathsf{ct}_1, \mathsf{ct}_2)$ *is an encryption of* $\Delta m_1 m_2$ *with error* $u \cdot e_\times = u \cdot (\frac{1}{\Delta} e_\otimes + e_{\mathsf{fresh}})$ *where*

$$e_\otimes = \Delta(m_1 e_2 + m_2 e_1) + u \cdot e_1 e_2, \qquad e_{\mathsf{fresh}} = \frac{e_{\mathsf{KS}}}{\Delta} + e_{\mathsf{RS}}.$$

Furthermore, assume $\|m_1\|_\infty^{\mathsf{can}}, \|m_2\|_\infty^{\mathsf{can}} \le 1$ *and* $\|e_1\|_\infty^{\mathsf{can}}, \|e_2\|_\infty^{\mathsf{can}} \le E$*. Then*

$$\|e_\times\|_\infty^{\mathsf{can}} \le 2E + \frac{\|u\|_\infty^{\mathsf{can}}}{\Delta} E^2 + E_{\mathsf{fresh}}, \qquad E_{\mathsf{fresh}} := \frac{1}{\Delta} E_{\mathsf{KS}} + E_{\mathsf{RS}}.$$

In particular, $\Delta \ge \|u\|_\infty^{\mathsf{can}} E$ *gives* $\|e_\times\|_\infty^{\mathsf{can}} \le 3E + E_{\mathsf{fresh}}$*.*

Proof. Writing $(d_0, d_1, d_2) = \mathsf{ct}_1 \otimes \mathsf{ct}_2$, we have

$$d_0 + d_1 \cdot \mathsf{sk} + d_2 \cdot \mathsf{sk}^2 = (\Delta m_1 + ue_1)(\Delta m_2 + ue_2) = \Delta^2 m_1 m_2 + ue_\otimes \pmod{Q}.$$

Focused relinearization adds ue_{KS} by Proposition 8. Applying Proposition 7 gives $e_\times = e_\otimes/\Delta + e_{\mathsf{KS}}/\Delta + e_{\mathsf{RS}}$. Hence

$$\|e_\times\|_\infty^{\mathsf{can}} \le 2E + \frac{\|u\|_\infty^{\mathsf{can}}}{\Delta} E^2 + E_{\mathsf{fresh}}.$$

If $\Delta \ge \|u\|_\infty^{\mathsf{can}} E$, the quadratic term is at most E.

Remark 2 (Error in background slots). In the focused CKKS setting, the intended messages $m_1, m_2 \in \mathcal{K}$ have $\sigma_j(m_1) = \sigma_j(m_2) = 0$ for each background slot $j \in [N/2] \setminus J_F$. Once we have a bound $|\sigma_j(e_1)|, |\sigma_j(e_2)| \le E_B$, the multiplication error on the background slot is

$$|\sigma_j(e_\times)| \le \frac{1}{\Delta}|\sigma_j(u \cdot e_1 e_2)| + |\sigma_j(e_{\mathsf{fresh}})| \le \frac{1}{\Delta}\,\|u\|_\infty^{\mathsf{can}} \cdot E_B^2 + E_{\mathsf{fresh}} < E_B,$$

provided $\Delta/2 > \|u\|_\infty^{\mathsf{can}}\, E_B$ and $E_{\mathsf{fresh}} < E_B/2$. Hence, with enough headroom in the scaling factor, the error in the background slots remains stable.

Meanwhile, an automorphism may change the unit, changing the error profile.

Proposition 12 (Automorphism with focused error). *Suppose* $\mathsf{ct} = (b, a) \in \mathcal{R}_Q^2$ *is an encryption of* Δm *with error* ue*. The homomorphic automorphism map for automorphism* $\kappa : \mathcal{K} \to \mathcal{K}$ *is given as*

$$\mathsf{Aut}_\kappa(\mathsf{ct}) := \mathsf{KS}^{[\kappa(u)]}_{\kappa(\mathsf{sk})\to\mathsf{sk}}((\kappa(b), \kappa(a)))$$

which encrypts $\Delta\kappa(m)$ *with error* $\kappa(u) \cdot (\kappa(e) + e_{\mathsf{KS}})$*. Furthermore, if* $\|e\|_\infty^{\mathsf{can}} \le E$*, one has high-probability bound*

$$\|\kappa(e) + e_{\mathsf{KS}}\|_\infty^{\mathsf{can}} \le E + E_{\mathsf{KS}}.$$

Proof. Applying κ to the RLWE equation gives

$$\kappa(b) + \kappa(a) \cdot \kappa(\mathsf{sk}) = \Delta\kappa(m) + \kappa(u)\kappa(e) \pmod{Q}.$$

Now apply Proposition 8 with unit $\kappa(u)$. Since κ permutes the canonical embeddings up to complex conjugation, $\|\kappa(e)\|_\infty^{\mathsf{can}} = \|e\|_\infty^{\mathsf{can}}$, giving the stated bound.

In practice, we restrict our attention to automorphisms fixing u, i.e., $\kappa(u) = u$, which preserve the error form. We write $\mathsf{Rot}_j^{[u]}$ for the above focused homomorphic evaluation of κ_j. For instance, the subring-based construction gives a unit $u \in \mathcal{R}'$ with automorphisms $\kappa \in \mathsf{Gal}(\mathcal{R}/\mathcal{R}')$ fixing u, and such automorphisms rotate the focus slots. Complex conjugation also fixes u whenever the focal unit is real, so focused homomorphic conjugation applies.

4.4 Linear Transformation and Polynomial Evaluation

Polynomial evaluation carries over directly to focused CKKS. A general linear transformation, however, is difficult to instantiate, since rotations mixing the focus and background slots do not fix u. Instead, we present linear transformations acting within each coset of slots, such as those supported on the focus slots.

Linear transformation. Let k divide $N/2$, and set $d = N/(2k)$. Let $\mathcal{R}' = \mathbb{Z}[Y]/(Y^{2k}+1)$, embedded in $\mathcal{R}$ by $Y = X^d$, and suppose the unit u lies in $\mathcal{R}'$. Since $5^k \equiv 1 \pmod{4k}$, κ_k fixes Y and hence u. Then the focused rotations $\mathsf{Rot}_{kt}^{[u]}$ preserve the error shape, and the rotations ρ^{kt} permute each coset $r + k[d]$ of slot indices cyclically. Hence we can evaluate linear maps that act on each coset separately, analogously to the one-dimensional transformations of [HS18, Sec. 3.1] with hypercolumns replaced by these cosets.

Lemma 7 (Focused linear transformation in each coset). *Fix linear maps $A_r : \mathbb{C}^d \to \mathbb{C}^d$ for $r \in [k]$, and let*

$$A = \sum_{r\in[k]} A_r \otimes \boldsymbol{e}_r \boldsymbol{e}_r^\top : \mathbb{C}^{N/2} \to \mathbb{C}^{N/2},$$

which applies A_r to the entries $(v_{ki+r})_{i\in[d]}$ for each $r \in [k]$. Suppose ct encrypts $\Delta\sigma^{-1}(\boldsymbol{v})$ with error ue. Then there is an operation

$$\mathsf{Linear}_A^{[u]} : \mathcal{R}_Q^2 \to \mathcal{R}_{Q/\Delta}^2$$

whose output encrypts $\Delta\sigma^{-1}(A\boldsymbol{v})$ with error ue', consuming one level. Let α bound the absolute values of the entries of A. Then e' satisfies the high-probability bound

$$\|e'\|_\infty^{\mathsf{can}} \le d\Big(\alpha + \frac{\|u\|_\infty^{\mathsf{can}}}{\Delta} E_{\mathsf{rnd}}\Big)\big(\|e\|_\infty^{\mathsf{can}} + E_{\mathsf{KS}}\big) + d\,\|\boldsymbol{v}\|_\infty E_{\mathsf{rnd}} + E_{\mathsf{RS}}.$$

Proof. For $t \in [d]$, let $\boldsymbol{a}_t \in \mathbb{C}^{N/2}$ have entries $(\boldsymbol{a}_t)_{ki+r} = (A_r)_{i,(i+t) \bmod d}$. Since $(\rho^{kt}\boldsymbol{v})_{ki+r} = v_{k((i+t) \bmod d)+r}$, we have $A\boldsymbol{v} = \sum_{t\in[d]} \boldsymbol{a}_t \odot \rho^{kt}(\boldsymbol{v})$. Set $a_t = \sigma^{-1}(\boldsymbol{a}_t)$, encode the diagonals, and evaluate

$$\begin{aligned}\mathsf{pt}_t &= \mathsf{Ecd}^{[u]}(a_t; \Delta),\\ \mathsf{Linear}_A^{[u]}(\mathsf{ct}) &= \mathsf{RS}_\Delta^{[u]}\Big(\sum_{t\in[d]} \mathsf{pt}_t \cdot \mathsf{Rot}_{kt}^{[u]}(\mathsf{ct})\Big).\end{aligned}$$

Each rotation fixes u. Write $\mathsf{pt}_t = \Delta a_t + u g_t$, and suppose the rotated ciphertexts decrypt as $\Delta m_t + u f_t$, where $m_t = \sigma^{-1}(\rho^{kt}\boldsymbol{v})$. By Proposition 10, summing before a single focused rescale gives error ue' with

$$e' = \sum_{t\in[d]} \Big(a_t f_t + m_t g_t + \frac{u}{\Delta} g_t f_t\Big) + e_{\mathsf{RS}}.$$

Here $\|a_t\|_\infty^{\mathsf{can}} \le \alpha$ and $\|m_t\|_\infty^{\mathsf{can}} = \|\boldsymbol{v}\|_\infty$. As rotation preserves the canonical norm, $\|f_t\|_\infty^{\mathsf{can}} \le \|e\|_\infty^{\mathsf{can}} + E_{\mathsf{KS}}$, and $\|g_t\|_\infty^{\mathsf{can}} \le E_{\mathsf{rnd}}$ by Proposition 4. Bounding each term with $\|e_{\mathsf{RS}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{RS}}$ gives the claim. The single rescale by Δ consumes one level.

Polynomial evaluation. For a polynomial $P \in \mathbb{R}[X]$ evaluated by a homomorphic circuit $\mathsf{PolyEval}_P$ of multiplicative depth D, we write

$$\mathsf{PolyEval}_P^{[u]} : \mathcal{R}_Q^2 \to \mathcal{R}_{Q/\Delta^D}^2$$

for the same circuit with each operation replaced by its focused counterpart. The following lemma treats the existing error of the input as part of the message, so it applies to any input ciphertext, focused or not.

Lemma 8 (Focused polynomial evaluation). *Suppose* $\mathsf{ct} \in \mathcal{R}_Q^2$ *encrypts* $\Delta(m+e)$. *Then* $\mathsf{PolyEval}_P^{[u]}(\mathsf{ct}) \in \mathcal{R}_{Q/\Delta^D}^2$ *encrypts*

$$\Delta P(m+e) + ue'$$

for some $e' \in \mathcal{K}$. *Moreover, if* $\|m+e\|_\infty^{\mathsf{can}} \le B$, *there is a constant* $C_{P,B}$, *depending only on* P, *its circuit, and* B, *such that if* $\Delta \ge \|u\|_\infty^{\mathsf{can}} E_{\mathsf{fresh}}$, *then* $\|e'\|_\infty^{\mathsf{can}} \le C_{P,B} E_{\mathsf{fresh}}$.

Proof. We show that each intermediate ciphertext encrypts $\Delta x + ue_x$, where x is the exact intermediate value in the evaluation of P at $m+e$.

Initially, $x = m+e$ and $e_x = 0$. It clearly holds for summing up ciphertexts and reducing by a smaller modulus. Adding a constant adds its focused encoding error ue_{Ecd} with $\|e_{\mathsf{Ecd}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{rnd}} \le E_{\mathsf{fresh}}$, which only contributes a fresh error. Meanwhile, the plaintext product and multiplication need some attention. By Propositions 10 and 11, for inputs $\Delta x_1 + ue_1$ and $\Delta x_2 + ue_2$, where the first may be a plaintext, both give

$$\Delta x + ue_x = \Delta x_1 x_2 + u\Big(x_1 e_2 + x_2 e_1 + \frac{u}{\Delta} e_1 e_2 + e_{\mathsf{fresh}}\Big),$$

where $e_{\mathsf{fresh}} = e_{\mathsf{RS}}$ for the plaintext product and $e_{\mathsf{fresh}} = e_{\mathsf{KS}}/\Delta + e_{\mathsf{RS}}$ for multiplication. By induction over the homomorphic operations, every intermediate ciphertext encrypts $\Delta x + ue_x$. At the output, $x = P(m+e)$, which gives the claim with $e' = e_x$.

For the bound, we show inductively that $\|e_x\|_\infty^{\mathsf{can}} \le C_x E_{\mathsf{fresh}}$ for a constant C_x depending only on P, its circuit, and B, and bound each term of e_x above. The cross terms $x_1 e_2$ and $x_2 e_1$ scale the input errors by $\|x_i\|_\infty^{\mathsf{can}}$, which is bounded in terms of P and B. The quadratic term $\frac{u}{\Delta} e_1 e_2$ is at most $C_{x_1} C_{x_2} E_{\mathsf{fresh}}$, since $\|u\|_\infty^{\mathsf{can}} E_{\mathsf{fresh}} \le \Delta$. The fresh error e_{fresh} is at most E_{fresh}. Taking $C_{P,B}$ to be the constant at the output completes the proof.

Remark 3. The constant $C_{P,B}$ also appears in ordinary CKKS, where the same argument holds by taking $u = 1$. Focusing only multiplies the fresh errors by u, at the cost of the scale requirement $\Delta \ge \|u\|_\infty^{\mathsf{can}} E_{\mathsf{fresh}}$ for the quadratic terms. A larger scale factor may yield a smaller constant than $C_{P,B}$, since it divides the quadratic term on each multiplication.

For a focused input, the existing error also stays focused through P.

Corollary 2. *Suppose* $\mathsf{ct} \in \mathcal{R}_Q^2$ *encrypts* $\Delta m + ue_{\mathsf{in}}$*, and* $\|m+x\|_\infty^{\mathsf{can}} \le B$ *for every* x *on the segment from* 0 *to* ue_{in}/Δ*. Then* $\mathsf{PolyEval}_P^{[u]}(\mathsf{ct})$ *encrypts* $\Delta P(m) + u(e''+e')$*, where* e' *is as in Lemma 8 and* $\|e''\|_\infty^{\mathsf{can}} \le L_{P,B}\,\|e_{\mathsf{in}}\|_\infty^{\mathsf{can}}$ *for* $L_{P,B} = \max_{t\in\mathbb{C},\,|t|\le B}|P'(t)|$.

Proof. Apply Lemma 8 with $e = ue_{\mathsf{in}}/\Delta$. Since P acts slotwise, $|\sigma_j(P(m+e)-P(m))| \le L_{P,B}|\sigma_j(e)| = L_{P,B}|\sigma_j(u)||\sigma_j(e_{\mathsf{in}})|/\Delta$ for every slot j. Hence $\Delta(P(m+e)-P(m)) = ue''$ with $\|e''\|_\infty^{\mathsf{can}} \le L_{P,B}\,\|e_{\mathsf{in}}\|_\infty^{\mathsf{can}}$.

5 Bootstrapping for Focused CKKS

We consider bootstrapping in the order

$$\mathsf{S2C} \to \mathsf{ModRaise} \to \mathsf{C2S} \to \mathsf{EvalMod},$$

starting and ending with messages in the focus slots. Focused error gives high precision in selected slots, but this advantage does not directly give small error in the coefficients. We therefore organize S2C and C2S so that most of their linear operations act on the focus slots, and change the error profile when passing to or from the coefficient representation. We use the focused linear transformation of Lemma 7. We note that the bootstrapping can also begin with ModRaise, in the order $\mathsf{ModRaise} \to \mathsf{C2S} \to \mathsf{EvalMod} \to \mathsf{S2C}$, if C2S and S2C are evaluated at high precision.

5.1 Slot layout

Take the subring $\mathcal{R}'$ of Section 4.4 with $k \ge 2$. The subring construction gives a real focal unit $u \in (\mathcal{R}')^\times$ with common embedding value $u_F \in (0,1)$ on the focus slots, whose indices form $J_F = k[d]$.

Given a vector $\boldsymbol{z} \in \mathbb{C}^d$ for the focus slots, we pad zeros in its background slots to obtain $\iota_k(\boldsymbol{z}) \in \mathbb{C}^{N/2}$, so that

$$(\iota_k(\boldsymbol{z}))_j = \begin{cases} z_i, & j = ki, \\ 0, & \text{otherwise.} \end{cases}$$

Since $u \in \mathcal{R}'$, Lemma 7 applies. For a single map $A : \mathbb{C}^d \to \mathbb{C}^d$, we write $\mathsf{Linear}_A^{[u]}$ for $\mathsf{Linear}_{A\otimes \boldsymbol{e}_0\boldsymbol{e}_0^\top}^{[u]}$. It maps an encryption of $\Delta\sigma^{-1}(\iota_k(\boldsymbol{z}))$ to that of $\Delta\sigma^{-1}(\iota_k(A\boldsymbol{z}))$, and discards the background of the input.

Remark 4. For a fixed map, ring parameters, and message and error bounds, the subring construction permits p-bit focus-slot precision with $\log\Delta = p/k + O(1)$. Rescaling therefore consumes fewer modulus bits than the $p + O(1)$ required by ordinary CKKS.

5.2 Encryption convention

Throughout this section, "bounded" means bounded a priori in canonical norm; we leave these bounds implicit. For any $\boldsymbol{v} \in \mathbb{C}^{N/2}$, saying that ct encrypts the slot vector $\boldsymbol{v}$ at scale Δ means that, for some bounded error ϵ,

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) = \Delta\sigma^{-1}(\boldsymbol{v}) + \epsilon \pmod{Q}.$$

We say the error is *focused* when $\epsilon = ue$ for some bounded e. Thus a ciphertext encrypting $\iota_k(\boldsymbol{z})$ with focused error satisfies

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) = \Delta\sigma^{-1}(\iota_k(\boldsymbol{z})) + ue \pmod{Q}. \tag{5}$$

The intended background values are zero; their decoded values contain error. The focused error profile allows larger error in the background slots.

Throughout the homomorphic constructions, Δ is an integer with $\Delta \geq \|u\|_\infty^{\mathsf{can}}$, each rescale factor divides its input modulus, and the claimed decoded values fit their output moduli.

5.3 Changing focus and interleaving

For $m = \sigma^{-1}(\iota_k(\boldsymbol{z}))$, we have $um = u_F m$, so

$$u\big(u_F^{-1}\Delta m + e\big) = \Delta m + ue. \tag{6}$$

Thus multiplying a ciphertext by u changes its scale from $u_F^{-1}\Delta$ to Δ and focuses its error, while preserving the intended slot values. Multiplication by u^{-1} reverses this conversion.

Conversion with a fully packed ciphertext. We consider conversion between one ciphertext using all dk slots at scale $u_F^{-1}\Delta$ with ordinary error, and k ciphertexts whose messages are zero outside the focus slots, at scale Δ with focused error.

Define the row-major map $\mathcal{J} : \mathbb{C}^{d\times k} \to \mathbb{C}^{dk}$ by

$$\mathcal{J}\left(\begin{bmatrix} z_{0,0} & \cdots & z_{0,k-1} \\ \vdots & \ddots & \vdots \\ z_{d-1,0} & \cdots & z_{d-1,k-1} \end{bmatrix}\right) = (z_{0,0}, \ldots, z_{0,k-1},\ \ldots,\ z_{d-1,0}, \ldots, z_{d-1,k-1}).$$

It reads the entries row by row, interleaving the columns: $(\mathcal{J}(\boldsymbol{Z}))_{ik+j} = z_{i,j}$.

We now introduce $\mathsf{Interleave}$ and $\mathsf{Deinterleave}$, which convert between ciphertexts encrypting the zero-padded columns of $\boldsymbol{Z} \in \mathbb{C}^{d\times k}$, and a ciphertext encrypting $\mathcal{J}(\boldsymbol{Z})$.

$$\mathsf{Interleave}(\{\mathsf{ct}_j\}_{j\in[k]}) = \sum_{j\in[k]} \mathsf{Rot}_{-j}(u^{-1}\cdot \mathsf{ct}_j),$$

$$\mathsf{Deinterleave}(\mathsf{ct}) = \{u \cdot \mathsf{RS}_{\lfloor u_F^{-1}\Delta\rceil}(\mathsf{mk}\cdot \mathsf{Rot}_j(\mathsf{ct}))\}_{j\in[k]},$$

where $\mathsf{mk} = \mathsf{Ecd}_{\mathsf{slot}}(\iota_k(1,\cdots,1); \lfloor u_F^{-1}\Delta\rceil) \in \mathcal{R}$.

Proposition 13 (Interleave and Deinterleave). *Suppose* $\boldsymbol{Z} = [\boldsymbol{z}_0 \mid \cdots \mid \boldsymbol{z}_{k-1}]$ *and* $\boldsymbol{w} = \mathcal{J}(\boldsymbol{Z})$.

1. *If each* ct_j *encrypts* $\iota_k(\boldsymbol{z}_j)$ *at scale* Δ *with focused error, then the ciphertext* $\mathsf{ct}' = \mathsf{Interleave}(\{\mathsf{ct}_j\}_{j\in[k]})$ *encrypts* $\boldsymbol{w}$ *at scale* $u_F^{-1}\Delta$ *with ordinary error.*
2. *If* ct' *encrypts* $\boldsymbol{w}$ *at scale* $u_F^{-1}\Delta$ *with ordinary error and* $\mathsf{Deinterleave}(\mathsf{ct}') = \{\mathsf{ct}_j\}_{j\in[k]}$, *then each* ct_j *encrypts* $\iota_k(\boldsymbol{z}_j)$ *at scale* Δ *with focused error.*

Proof. Set $m_j = \sigma^{-1}(\iota_k(\boldsymbol{z}_j))$ and $m = \sigma^{-1}(\boldsymbol{w})$; decryption identities are modulo the output modulus. For interleaving, write $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_j) = \Delta m_j + ue_j$. Since $u^{-1}m_j = u_F^{-1}m_j$ and $\sum_j \kappa_{-j}(m_j) = m$,

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}') = u_F^{-1}\Delta m + e',$$

where e' is bounded as a sum of rotated input errors and key-switching errors.

For deinterleaving, set $\mathsf{mask}_F = \sigma^{-1}(\iota_k(1,\ldots,1))$. Since $\mathsf{mask}_F\kappa_j(m) = m_j$ and the mask scale equals the rescale divisor, masking and rescaling give decryption $u_F^{-1}\Delta m_j + r_j$. The error r_j is bounded by the canonical norm inequalities applied to the input, encoding, rotation, and rescaling errors. Finally, $um_j = u_F m_j$ gives

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_j) = \Delta m_j + ur_j,$$

proving the focused-error claim.

5.4 Linear transforms for S2C and C2S

The desired maps on slot vectors are

$$\mathsf{S2C} : \iota_k(m_0, m_k, \ldots, m_{(d-1)k}) \mapsto \boldsymbol{z},$$
$$\mathsf{C2S} : \hat{\boldsymbol{z}} \mapsto \iota_k(\hat{m}_0, \hat{m}_k, \ldots, \hat{m}_{(d-1)k}),$$

where

$$\boldsymbol{z} = \sigma\Big(\sum_{j\in[d]} m_{jk}X^{jk}\Big), \qquad \hat{\boldsymbol{z}} = \sigma\Big(\sum_{i\in[dk]} \hat{m}_i X^i\Big).$$

Here the polynomials are represented in $\mathbb{C}[X]/(X^{N/2} - \mathsf{i})$, so the displayed coefficients may be complex. The homomorphic operations should also change the scale: $\mathsf{S2C}$ goes from Δ with focused error to $u_F^{-1}\Delta$ with ordinary error, and $\mathsf{C2S}$ makes the reverse change. While $\mathsf{S2C}$ places the intended message only in coefficients of X^{jk}, $\mathsf{ModRaise}$ can introduce overflow in every coefficient, so $\mathsf{C2S}$ is defined on all dk coefficients.

For the block decomposition below, assume $k \mid d$.

Proposition 14 (Linear maps for S2C and C2S). *Denote* $\mathcal{L}_{\mathsf{S2C}} : \mathbb{C}^d \to \mathbb{C}^{dk}$, $\mathcal{L}_{\mathsf{C2S}} : \mathbb{C}^{dk} \to \mathbb{C}^d$ *to be*

$$\mathcal{L}_{\mathsf{S2C}}(\boldsymbol{m}) = (\mathcal{V}_d\boldsymbol{m} \mid \cdots \mid \mathcal{V}_d\boldsymbol{m}),$$
$$\mathcal{L}_{\mathsf{C2S}}(\boldsymbol{z}_0 \mid \cdots \mid \boldsymbol{z}_{k-1}) = \frac{1}{k}\sum_{j\in[k]} \mathcal{V}_d^{-1}\boldsymbol{z}_j.$$

Then, one has

$$\mathcal{L}_{\mathsf{S2C}}((m_0, m_k, \cdots, m_{(d-1)k})) = \sigma\Big(\sum_{j\in[d]} m_{jk} X^{jk}\Big),$$
$$\mathcal{L}_{\mathsf{C2S}}\Big(\sigma\big(\sum_{i\in[dk]} \hat{m}_i X^i\big)\Big) = (\hat{m}_0, \hat{m}_k, \cdots, \hat{m}_{(d-1)k}).$$

Proof. For a polynomial supported on powers of X^k, evaluation gives

$$\sum_i m_{ik}\zeta^{ik5^j} = \sum_i m_{ik}\zeta_{4d}^{i5^j},$$

which repeats with period d in j. This proves the first identity. For the second, the powers 5^{td}, $t \in [k]$, run through $1 + 4dt$ modulo $4dk$. Hence, for $j \in [d]$ and $i \in [dk]$,

$$\frac{1}{k}\sum_{t\in[k]} \zeta^{i5^{j+td}} = \begin{cases} \zeta_{4d}^{(i/k)5^j}, & k \mid i, \\ 0, & k \nmid i. \end{cases}$$

Averaging the k slot blocks therefore gives $\mathcal{V}_d(\hat{m}_0, \hat{m}_k, \ldots, \hat{m}_{(d-1)k})^\top$. Applying $\mathcal{V}_d^{-1}$ extracts the stated coefficients.

We perform most of each linear transform on zero-padded slot vectors with focused error. Thus the decomposition of $\mathcal{L}_{\mathsf{S2C}}$ ends with $\mathcal{J}$, where we remove the focus and interleave, and the decomposition of $\mathcal{L}_{\mathsf{C2S}}$ begins with $\mathcal{J}^{-1}$, where we separate the slots and gain focus. To undo the interleaving, we define the permutation $\pi : [d] \to [d]$ as

$$\pi(jk + i) = id/k + j, \quad i \in [k], j \in [d/k].$$

The following maps are involved in the decomposition.

1. The DFT map $\mathcal{T} : \mathbb{C}^d \to \mathbb{C}^d$ is given as $\mathcal{T} = \mathcal{P}_\pi \mathcal{V}_d$, so that
$$\mathcal{T}(\boldsymbol{m}) = \boldsymbol{z}, \qquad z_{\pi(j)} = \sum_{i\in[d]} m_i \zeta_{4d}^{i5^j}, \quad j \in [d].$$
Denote its inverse $\mathcal{T}^{-1} : \mathbb{C}^d \to \mathbb{C}^d$.
2. The rotating fan-out $\mathcal{O} : \mathbb{C}^d \to \mathbb{C}^{d\times k}$ and rotating fan-in $\mathcal{O}^* : \mathbb{C}^{d\times k} \to \mathbb{C}^d$ are given by
$$\mathcal{O}(\boldsymbol{z}) = [\,\boldsymbol{z} \mid \rho^{-d/k}(\boldsymbol{z}) \mid \cdots \mid \rho^{-(k-1)d/k}(\boldsymbol{z})\,],$$
$$\mathcal{O}^*([\,\boldsymbol{z}_0 \mid \cdots \mid \boldsymbol{z}_{k-1}\,]) = \sum_{j\in[k]} \rho^{jd/k}(\boldsymbol{z}_j).$$
3. Write $\boldsymbol{Z} \in \mathbb{C}^{d\times k}$ as an array of column-vector blocks:
$$\boldsymbol{Z} = \begin{bmatrix} \boldsymbol{z}_{0,0} & \cdots & \boldsymbol{z}_{0,k-1} \\ \vdots & & \vdots \\ \boldsymbol{z}_{k-1,0} & \cdots & \boldsymbol{z}_{k-1,k-1} \end{bmatrix}.$$

where $\boldsymbol{z}_{\ell,j} \in \mathbb{C}^{d/k}$ for $\ell, j \in [k]$. The block shuffle $\mathcal{E} : \mathbb{C}^{d\times k} \to \mathbb{C}^{d\times k}$ maps $\boldsymbol{Z}$ to $\boldsymbol{W}$ whose blocks satisfy

$$\boldsymbol{w}_{\ell,j} = \boldsymbol{z}_{\ell,(\ell-j) \bmod k}, \qquad \ell, j \in [k].$$

It permutes blocks across columns within each block row.

Now, we give the decomposition.

Lemma 9 (Decomposition for S2C and C2S). *We have*

$$\mathcal{L}_{\mathsf{S2C}} = \mathcal{J} \circ \mathcal{E} \circ \mathcal{O} \circ \mathcal{T},$$
$$\mathcal{L}_{\mathsf{C2S}} = \tfrac{1}{k}\mathcal{T}^{-1} \circ \mathcal{O}^* \circ \mathcal{E} \circ \mathcal{J}^{-1}.$$

Proof. Let $R : \mathbb{C}^d \to \mathbb{C}^{dk}$ be $R(\boldsymbol{z}) = (\boldsymbol{z} \mid \cdots \mid \boldsymbol{z}) \in \mathbb{C}^{dk}$, and $S : \mathbb{C}^{dk} \to \mathbb{C}^d$ be $S(\boldsymbol{z}_0 \mid \cdots \mid \boldsymbol{z}_{k-1}) = \sum_{j\in[k]} \boldsymbol{z}_j$. Then, $\mathcal{L}_{\mathsf{S2C}} = R\mathcal{V}_d$ and $\mathcal{L}_{\mathsf{C2S}} = \frac{1}{k}\mathcal{V}_d^{-1}S$ by definition.

First, we show $R = \mathcal{J}\mathcal{E}\mathcal{O}\mathcal{P}_\pi$, so that $\mathcal{L}_{\mathsf{S2C}} = R\mathcal{V}_d = \mathcal{J}\mathcal{E}\mathcal{O}\mathcal{T}$. Denoting $\mathcal{P}_\pi \boldsymbol{z} = (\boldsymbol{z}_0 \mid \cdots \mid \boldsymbol{z}_{k-1})$ as column vectors,

$$\boldsymbol{z} \xrightarrow{\mathcal{P}_\pi} \begin{bmatrix} \boldsymbol{z}_0 \\ \vdots \\ \boldsymbol{z}_{k-1} \end{bmatrix} \xrightarrow{\mathcal{O}} \begin{bmatrix} \boldsymbol{z}_0 & \boldsymbol{z}_{k-1} & \cdots & \boldsymbol{z}_1 \\ \boldsymbol{z}_1 & \boldsymbol{z}_0 & \cdots & \boldsymbol{z}_2 \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{z}_{k-1} & \boldsymbol{z}_{k-2} & \cdots & \boldsymbol{z}_0 \end{bmatrix} \xrightarrow{\mathcal{E}} \begin{bmatrix} \boldsymbol{z}_0 & \boldsymbol{z}_1 & \cdots & \boldsymbol{z}_{k-1} \\ \boldsymbol{z}_0 & \boldsymbol{z}_1 & \cdots & \boldsymbol{z}_{k-1} \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{z}_0 & \boldsymbol{z}_1 & \cdots & \boldsymbol{z}_{k-1} \end{bmatrix} \xrightarrow{\mathcal{J}} \begin{bmatrix} \boldsymbol{z} \\ \vdots \\ \boldsymbol{z} \end{bmatrix} = R\boldsymbol{z},$$

since row-wise flattening of $\begin{bmatrix} \boldsymbol{z}_0 & \cdots & \boldsymbol{z}_{k-1} \end{bmatrix}$ is $\boldsymbol{z}$:

$$z_{jk+i} = (\mathcal{P}_\pi \boldsymbol{z})_{id/k+j} = (\boldsymbol{z}_i)_j.$$

For $\mathcal{L}_{\mathsf{C2S}}$, the following maps are adjoints under the standard inner product over $\mathbb{C}$, with the matrices flattened:

$$R^* = S, \quad \mathcal{P}_\pi^* = \mathcal{P}_\pi^{-1}, \quad (\mathcal{O})^* = \mathcal{O}^*, \quad \mathcal{E}^* = \mathcal{E}, \quad \mathcal{J}^* = \mathcal{J}^{-1}.$$

Hence, $S = \mathcal{P}_\pi^{-1}\mathcal{O}^*\mathcal{E}\mathcal{J}^{-1}$, so $\mathcal{L}_{\mathsf{C2S}} = \frac{1}{k}\mathcal{V}_d^{-1}S = \frac{1}{k}\mathcal{T}^{-1}\mathcal{O}^*\mathcal{E}\mathcal{J}^{-1}$.

5.5 Homomorphic operations for S2C and C2S

The following homomorphic operations correspond to the linear maps.

$$\mathsf{DFT}(\mathsf{ct}) = \mathsf{Linear}_{\mathcal{T}}^{[u]}(\mathsf{ct}), \qquad \mathsf{iDFT}(\mathsf{ct}) = \mathsf{Linear}_{\mathcal{T}^{-1}}^{[u]}(\mathsf{ct}),$$
$$\mathsf{RotFanout}(\mathsf{ct}) = (\mathsf{Rot}_{-jd}^{[u]}(\mathsf{ct}))_{j\in[k]},$$
$$\mathsf{RotFanin}(\{\mathsf{ct}_j\}_{j\in[k]}) = \sum_{j\in[k]} \mathsf{Rot}_{jd}^{[u]}(\mathsf{ct}_j),$$
$$\mathsf{Shuffle}(\{\mathsf{ct}_j\}_{j\in[k]}) = \Big\{\mathsf{RS}_{\Delta}^{[u]}\big(\sum_{\ell\in[k]} \mathsf{bl}_\ell \cdot \mathsf{ct}_{\ell-j}\big)\Big\}_{j\in[k]},$$

where ciphertext indices are taken modulo k. For $\ell \in [k]$, define $\boldsymbol{b}_\ell \in \{0,1\}^d$ by

$$(\boldsymbol{b}_\ell)_i = \begin{cases} 1, & \ell d/k \le i < (\ell+1)d/k, \\ 0, & \text{otherwise}, \end{cases} \qquad i \in [d],$$

and encode

$$\mathsf{bl}_\ell = \mathsf{Ecd}^{[u]}(\sigma^{-1}(\iota_k(\boldsymbol{b}_\ell)); \Delta).$$

These are plaintext masks constructed with the focused encoder of Definition 6.

By Lemma 7, DFT and iDFT evaluate $\mathcal{T}$ and $\mathcal{T}^{-1}$ on the focus slots. The physical rotation index is k times the rotation index in $\mathbb{C}^d$, so $\mathsf{RotFanout}$ and $\mathsf{RotFanin}$ evaluate $\mathcal{O}$ and $\mathcal{O}^*$. All these operations preserve zero background messages and focused error.

Lemma 10 ($\mathsf{Shuffle}$ evaluates $\mathcal{E}$). *Suppose each ct_j encrypts $\iota_k(\boldsymbol{z}_j)$ at scale Δ with focused error, and let $\{\mathsf{ct}'_j\} = \mathsf{Shuffle}(\{\mathsf{ct}_j\})$. Then each ct'_j encrypts $\iota_k(\boldsymbol{w}_j)$ at scale Δ with focused error, where $[\boldsymbol{w}_0 \mid \cdots \mid \boldsymbol{w}_{k-1}] = \mathcal{E}([\boldsymbol{z}_0 \mid \cdots \mid \boldsymbol{z}_{k-1}])$.*

Proof. At the level of intended slot values,

$$\boldsymbol{w}_j = \sum_{\ell \in [k]} \boldsymbol{b}_\ell \odot \boldsymbol{z}_{\ell - j},$$

which selects block ℓ from column $\ell - j$ as required by $\mathcal{E}$. Zero padding commutes with these sums and coordinatewise products. The product and focused rescale formulas give output error of the form ue', including the mask encoding errors.

Theorem 2 (Correctness of S2C). *Let $\boldsymbol{m} = (m_0, m_k, \ldots, m_{(d-1)k})$. Given a ciphertext ct encrypting $\iota_k(\boldsymbol{m})$ at scale Δ with focused error, $\mathsf{Interleave} \circ \mathsf{Shuffle} \circ \mathsf{RotFanout} \circ \mathsf{DFT}(\mathsf{ct})$ encrypts*

$$u_F^{-1}\Delta \cdot \sum_{j \in [d]} m_{kj} X^{kj}$$

with ordinary error.

Proof. Set $\boldsymbol{Z} = \mathcal{E}\mathcal{O}\mathcal{T}\boldsymbol{m}$ and

$$\{\mathsf{ct}'_j\}_{j \in [k]} = \mathsf{Shuffle} \circ \mathsf{RotFanout} \circ \mathsf{DFT}(\mathsf{ct}).$$

Each ct'_j encrypts $\iota_k(\boldsymbol{Z}_{*j})$ at scale Δ with focused error. By Proposition 13, applying $\mathsf{Interleave}$ gives a ciphertext encrypting $\mathcal{J}\mathcal{E}\mathcal{O}\mathcal{T}\boldsymbol{m} = \mathcal{L}_{\mathsf{S2C}}(\boldsymbol{m})$ at scale $u_F^{-1}\Delta$ with ordinary error. Its intended plaintext polynomial is

$$u_F^{-1}\Delta\sigma^{-1}(\mathcal{L}_{\mathsf{S2C}}(\boldsymbol{m})) = u_F^{-1}\Delta \sum_{j \in [d]} m_{kj} X^{kj}$$

by Proposition 14.

Theorem 3 (Correctness of C2S). *Given a ciphertext* ct *encrypting*

$$u_F^{-1}\Delta \cdot \sum_{i\in[dk]} m_i X^i$$

with ordinary error, the ciphertext

$$\mathsf{ct}' = \tfrac{1}{k}\mathsf{iDFT} \circ \mathsf{RotFanin} \circ \mathsf{Shuffle} \circ \mathsf{Deinterleave}(\mathsf{ct})$$

encrypts the zero-padded vector $\iota_k(m_0, m_k, \ldots, m_{(d-1)k})$ *at scale* Δ *with focused error. Here* $\frac{1}{k}\mathsf{iDFT}$ *denotes the inverse transform with* $1/k$ *absorbed into its plaintext diagonals.*

Proof. Set

$$\boldsymbol{z} = \sigma\Big(\sum_{i\in[dk]} m_i X^i\Big).$$

Thus ct encrypts the slot vector $\boldsymbol{z}$ at scale $u_F^{-1}\Delta$. By Proposition 13, $\mathsf{Deinterleave}$ gives ciphertexts encrypting $\iota_k((\mathcal{J}^{-1}\boldsymbol{z})_{*j})$ at scale Δ with focused error. The remaining operations give a ciphertext encrypting

$$\iota_k\big(\tfrac{1}{k}\mathcal{T}^{-1}\mathcal{O}^*\mathcal{E}\mathcal{J}^{-1}\boldsymbol{z}\big) = \iota_k(\mathcal{L}_{\mathsf{C2S}}(\boldsymbol{z}))$$

at scale Δ with focused error. The result follows from Proposition 14.

5.6 Optimizing C2S modulus consumption

The above $\mathsf{C2S}$ procedure requires rescaling by $\lfloor u_F^{-1}\Delta \rceil$ in $\mathsf{Deinterleave}$, which is potentially large. To remove this rescale, we fuse $\mathsf{ModRaise}$ with $\mathsf{Deinterleave}$ inside $\mathsf{C2S}$.

Multiplying a lift before rounding. Given $\mathsf{ct} \in \mathcal{R}_q^2$, take its centered lift $\hat{\mathsf{ct}} = (b, a) \in \mathcal{R}^2$ and write $\hat{m} = b + a \cdot \mathsf{sk}$ for its exact decryption before reduction modulo q. For a polynomial $c \in \mathcal{K}$, coefficient-wise rounding gives

$$\lfloor cb \rceil + \lfloor ca \rceil \cdot \mathsf{sk} = c\hat{m} + e_{\mathsf{lift}},$$

where

$$e_{\mathsf{lift}} = (\lfloor cb \rceil - cb) + (\lfloor ca \rceil - ca) \cdot \mathsf{sk}.$$

This has the same form as rescale error. Under the rounding-error model used for Lemma 4, it satisfies the high-probability bound $\|e_{\mathsf{lift}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{RS}}$. Embedding the resulting ciphertext into a sufficiently large modulus Q therefore gives an encryption of $c\hat{m}$ with rounding error e_{lift}. This permits applying a mask directly to the lift without a separate rescale.

Fused Deinterleave. Let $\mathsf{mask}_F = \sigma^{-1}(\iota_k(1,\dots,1))$ be the ideal slot mask. Define

$$\mathsf{Deinterleave}_Q(\mathsf{ct}) = \{\tilde{\mathsf{ct}}_j\}_{j\in[k]}, \qquad \tilde{\mathsf{ct}}_j = u \cdot \mathsf{Rot}_j(\mathsf{ct}'_j),$$
$$\mathsf{ct}'_j = [\lfloor \kappa_{-j}(\mathsf{mask}_F) \cdot \hat{\mathsf{ct}} \rceil]_Q \in \mathcal{R}_Q^2,$$

where rounding is applied to both components of the centered lift $\hat{\mathsf{ct}}$. This operation retains modulus Q, avoiding the rescale by $\lfloor u_F^{-1}\Delta \rceil$ in ordinary deinterleaving. It computes $\mathcal{J}^{-1}$ on the lifted slot vector as follows.

Proposition 15 (Fused deinterleave). *Given* $\mathsf{ct} \in \mathcal{R}_q^2$*, take its centered lift* $\hat{\mathsf{ct}} = (b,a) \in \mathcal{R}^2$ *and set*

$$\hat{\boldsymbol{z}} = (u_F^{-1}\Delta)^{-1}\sigma(b + a\cdot \mathsf{sk}).$$

Thus $b + a\cdot\mathsf{sk} = u_F^{-1}\Delta\sigma^{-1}(\hat{\boldsymbol{z}})$ *exactly. Each output* $\tilde{\mathsf{ct}}_j$ *of* $\mathsf{Deinterleave}_Q(\mathsf{ct})$ *encrypts* $\iota_k((\mathcal{J}^{-1}\hat{\boldsymbol{z}})_{*j})$ *at scale* Δ *with focused error.*

Proof. Put $\boldsymbol{z}_j = (\hat{z}_j, \hat{z}_{j+k}, \dots, \hat{z}_{j+(d-1)k})$. The mask $\kappa_{-j}(\mathsf{mask}_F)$ selects the slots with indices congruent to j modulo k. Rotating by j then moves their values into the focus slots, so

$$\kappa_j\big(\kappa_{-j}(\mathsf{mask}_F)(b + a\cdot\mathsf{sk})\big) = u_F^{-1}\Delta\sigma^{-1}(\iota_k(\boldsymbol{z}_j)).$$

Hence $\mathsf{Rot}_j(\mathsf{ct}'_j)$ encrypts $\iota_k(\boldsymbol{z}_j)$ at scale $u_F^{-1}\Delta$ with ordinary error from rounding and key switching. Multiplication by u changes the scale to Δ and focuses this error by (6).

The modified C2S operation is

$$\tfrac{1}{k}\mathsf{iDFT} \circ \mathsf{RotFanin} \circ \mathsf{Shuffle} \circ \mathsf{Deinterleave}_Q(\mathsf{ct}).$$

Suppose the input lift satisfies

$$b + a\cdot\mathsf{sk} = u_F^{-1}\Delta m + qI + e.$$

Writing m_i and I_i for the coefficients in the complex coefficient representation, set

$$v_j = m_{jk} + \frac{u_F q}{\Delta} I_{jk}, \qquad j \in [d].$$

The output encrypts $\iota_k(\boldsymbol{v})$ at scale Δ with focused error, including the propagated input error e and the errors of the homomorphic operations. For C2S, we interpret the ordinary input at nominal scale q, so the focused linear transformations after $\mathsf{Deinterleave}_Q$ use nominal scale $u_F q$, with plaintext scales and rescale factors of this order. EvalMod uses the same scale, at which the output focus-slot values are

$$\tilde{v}_j = \frac{\Delta}{u_F q} m_{jk} + I_{jk}.$$

This reduces the usual q-scale by a factor of u_F without consuming modulus.

5.7 Cost of S2C and C2S

We use the fused $\mathsf{Deinterleave}_Q$ for C2S throughout this accounting.

Modulus accounting. The base modulus q is the modulus after S2C and before ModRaise. For p-bit precision, let $E_{\text{out}} \geq 1$ bound the ordinary error after S2C. Let $\delta_{\text{diff}} = \log_2(u_F q/\Delta)$ denote the gap in bits between the base modulus and the coefficient scale. Choosing the coefficient scale of order $2^p E_{\text{out}}$ gives, for fixed message range,

$$\log q = p + \log E_{\text{out}} + \delta_{\text{diff}} + O(1).$$

Thus the base modulus retains all p precision bits plus error and gap bits; the $1/k$ reduction applies to the modulus consumed by rescaling. The gap required by EvalMod is discussed below.

As in ordinary CKKS bootstrapping, C2S operates at the scale given by the base modulus: after the fused deinterleave, its input is read at scale $u_F q$ rather than Δ. With a single level for DFT or iDFT and one for Shuffle, the modulus changes are

$$q\Delta^2 \xrightarrow{\mathsf{S2C}} q, \qquad Q \xrightarrow{\mathsf{C2S}} Q/(u_F q)^2.$$

Hence S2C consumes $2\log\Delta$ and C2S consumes $2\log(u_F q)$ modulus bits, where $u_F q$ can be reduced by the choice of focal unit as discussed in Section 5.8. The k outputs of Shuffle are computed in parallel and consume one level. Rotations, interleaving, and fused deinterleaving consume no levels.

Key-switching count with a single-level DFT. For $d = bg$, baby-step/giant-step (BSGS) evaluation of either dense transform uses $b-1$ baby-step rotations and $g-1$ giant-step rotations [HS18, Sec. 6]. All products are summed before one focused rescale. Taking $b = 2^{\lfloor (\log d)/2 \rfloor}$ and $g = d/b$ gives $O(\sqrt{d})$ key switches. The remaining counts are

	S2C	C2S
DFT or iDFT	$b+g-2$	$b+g-2$
Fan-out or fan-in	$k-1$	$k-1$
Interleave or fused deinterleave	$k-1$	$k-1$
Shuffle	0	0
Total	$b+g+2k-4$	$b+g+2k-4$

Here each nontrivial rotation uses one key switch; the DFT or iDFT is evaluated once per direction.

ℓ-level DFT decomposition. Put $\mathcal{P}_{\text{in}} = \mathcal{P}_\pi \mathcal{P}_{\text{rev}}$. Using $\mathcal{P}_{\text{in}}\boldsymbol{m}$ as the external focus-slot order replaces the DFT matrix by

$$\mathcal{T}\mathcal{P}_{\text{in}}^{-1} = \mathcal{P}_\pi(\mathcal{V}_d \mathcal{P}_{\text{rev}})\mathcal{P}_\pi^{-1},$$

and the iDFT matrix by its inverse. The matrix $\mathcal{V}_d\mathcal{P}_{\mathrm{rev}}$ has the standard $\log d$ butterfly factors [CCS19]. The permutation π rearranges index bits, so each butterfly still pairs coordinates whose indices differ in one bit. Each factor therefore has at most three cyclic diagonals. The inverse transform returns the same external order, and slotwise EvalMod respects it, so this layout requires no separate homomorphic permutation.

For $1 \le \ell \le \log d$, group the butterfly factors into ℓ consecutive groups whose sizes differ by at most one, rescaling once per group. A group of r factors has $O(2^r)$ cyclic diagonals and admits BSGS evaluation with $O(2^{r/2})$ rotations. Thus each direction uses

$$O\Big(\ell d^{1/(2\ell)}\Big) + 2(k-1) \quad \text{key switches}, \qquad (\ell+1)\log \Delta \quad \text{modulus bits}.$$

The additional level is for Shuffle.

5.8 EvalMod in the High-Precision Setting

After C2S, each focus slot contains a message component scaled by $\Delta/(u_F q)$ plus an integer overflow. EvalMod removes this overflow and restores the message using the standard trigonometric approximation.

High precision with a large gap. For a real component $x = I + 2^{-\delta_{\mathrm{diff}}} m$, where $I \in \mathbb{Z}$ and $|m| \le 1$,

$$\frac{2^{\delta_{\mathrm{diff}}}}{2\pi} \sin(2\pi x) = m + O\big(m^3 2^{-2\delta_{\mathrm{diff}}}\big).$$

Choosing $\delta_{\mathrm{diff}} = p/2 + O(1)$ makes this error $O(2^{-p})$, at the cost of $p/2 + O(1)$ additional base-modulus bits. For a fixed overflow bound $|I| < K$, Han–Ki's interpolation bound implies that degree $D = O(K)$ suffices for error $O(2^{-2\delta_{\mathrm{diff}}})$ after restoring the message scale [HK20, Sec. 4.1, Eq. (3)]. A larger gap confines the input to narrower intervals around the integers, so the interpolation error decreases even at fixed degree. A fixed number r of cosine double-angle steps preserves this error order, since each step amplifies a small error by $O(1)$. Their evaluation uses $O(\sqrt{D} + r)$ non-scalar multiplications and $O(\log D) + r$ multiplicative depth per real component [HK20, Secs. 4.2–4.3]. Thus the interpolation degree and multiplicative depth are independent of p for fixed K and r.

For complex inputs, we use conjugation to separate the real and imaginary parts, apply EvalMod to each, and recombine.

Optimizing the base modulus. An inverse-sine correction removes the need for a gap growing with precision [LLL+21]:

$$\frac{2^{\delta_{\mathrm{diff}}}}{2\pi} \arcsin\big(\sin(2\pi x)\big) = m, \qquad 2^{-\delta_{\mathrm{diff}}}|m| < 1/4.$$

With a fixed margin from $1/4$, we can keep δ_{diff} constant and approximate the inverse sine to the desired precision. This saves the $p/2 + O(1)$ gap bits in the base modulus at the cost of evaluating an additional polynomial. At fixed gap, overflow bound, and double-angle count, both the trigonometric and inverse-sine approximation degrees grow with p.

Modulus savings from focusing. The C2S and EvalMod stages may use a stronger focal unit than S2C. Let $0 < E < q$ satisfy $\|e\|_\infty^{\mathsf{can}} \le E$ for each focused error ue in the computation. Both stages use scale $u_F q$, and the quadratic error term in Proposition 11 is at most $\|u\|_\infty^{\mathsf{can}} E^2/(u_F q)$, so we require

$$u_F q \ge E \, \|u\|_\infty^{\mathsf{can}}$$

to keep this term at most E. Stronger focusing decreases u_F but increases $\|u\|_\infty^{\mathsf{can}}$; for the subring construction of Proposition 3,

$$\log \|u\|_\infty^{\mathsf{can}} = -\frac{1}{k-1} \log u_F + O(1),$$

where the hidden constant depends on the ring. Thus we choose the focal unit so that $u_F q$ is of order $E \, \|u\|_\infty^{\mathsf{can}}$ while satisfying the inequality above. Substituting this relation gives

$$\log(u_F q) = \frac{1}{k} \log q + \left(1 - \frac{1}{k}\right) \log E + O(1).$$

With arithmetic scales of this order and fixed depth, C2S and EvalMod consume roughly $1/k$ of the modulus bits dependent on the precision; the error-dependent overhead remains.

For example, in the ideal focal-unit model, $\log q = 56$, $\log E = 16$, and $k = 2$ give $\log(u_F q) \approx 16 + (56 - 16)/2 = 36$. An eight-level evaluation at this scale would save $8(56 - 36) = 160$ modulus bits compared with evaluation at scale q.

6 Reducing Modulus Consumption in Focused Bootstrapping

We give another bootstrapping method, which begins with ModRaise so that the base modulus needs only about p/k bits instead of p. Following the high-precision bootstrapping of [CKSS25] based on discrete CKKS, we perform ModRaise and a low-precision C2S, and recover the overflow I by cleaning. We carry out the cleaning in the focused state, at k times the cost of the cleaning operations. For p bits of precision, the bootstrapping then consumes $O(p/k + \log p)$ bits of modulus, comparable to ordinary bootstrapping even at high precision.

6.1 Focused Cleaning for a Fully Packed Discrete Ciphertext

We first apply cleaning to a fully packed ciphertext with discrete messages. With focused operations, the focus slots gain precision while the background slots stay bounded.

Focused cleaning. Fix an order-M cleaning polynomial P for $\mathcal{D}$ as in Section 2.5. We define the *focused cleaning* as

$$\mathsf{Clean}^{[u]} = \mathsf{PolyEval}_P^{[u]}.$$

Following [CKSS25, Sec. 4.1], we measure precision in bits. We fix $0 < \epsilon < 1$ bounding the error on every slot, with $K_{P,\epsilon}\epsilon^M \le 1/2$, and set $c = \log K_{P,\epsilon}$. With $C = C_{P,B}$ for $B = \max_{b\in\mathcal{D}}|b| + \epsilon$, we also set $\eta' = \log(2CE_{\mathsf{fresh}}/\epsilon)$, which is independent of the precision.

Proposition 16 (Focused cleaning). *Let u be a p-bit focal unit for J_F and $\log\Delta \ge \log\|u\|_\infty^{\mathsf{can}} + \eta'$. Suppose ct encrypts $\Delta(m+e)$ with $\sigma_j(m) \in \mathcal{D}$ for every slot j, and $|\sigma_j(e)|$ is at most $2^{-p_{\mathsf{in}}} \le \epsilon$ for $j \in J_F$ and at most ϵ otherwise. Then $\mathsf{Clean}^{[u]}(\mathsf{ct})$ encrypts $\Delta(m+\tilde{e})$ with*

$$|\sigma_j(\tilde{e})| \le \begin{cases} 2^{-(M+1)p_{\mathsf{in}}+c} + 2^{-p}, & j \in J_F, \\ \epsilon, & \text{otherwise.} \end{cases}$$

In particular, the focus slots have precision at least $\min\big((M+1)p_{\mathsf{in}} - c,\ p\big) - 1$ bits.

Proof. Since $\|m+e\|_\infty^{\mathsf{can}} \le B$ and $\Delta \ge \|u\|_\infty^{\mathsf{can}} E_{\mathsf{fresh}}$, Lemma 8 shows that the output encrypts $\Delta P(m+e) + ue'$ with $\|e'\|_\infty^{\mathsf{can}} \le CE_{\mathsf{fresh}}$. Set $\tilde{e} = P(m+e) - m + ue'/\Delta$. As P has real coefficients, $\sigma_j(P(m+e)) = P(\sigma_j(m) + \sigma_j(e))$, which lies within $2^{-(M+1)p_{\mathsf{in}}+c}$ of $\sigma_j(m)$ for $j \in J_F$ and within $K_{P,\epsilon}\epsilon^{M+1} \le \epsilon/2$ otherwise. The remaining term is at most $|\sigma_j(u)|CE_{\mathsf{fresh}}/\Delta \le |\sigma_j(u)|\epsilon/(2\|u\|_\infty^{\mathsf{can}})$, which is at most 2^{-p} on the focus slots and $\epsilon/2$ elsewhere.

Iterated cleaning. Proposition 16 only assumes an encryption of $\Delta(m+e)$, and every slot stays within ϵ, so the focused cleaning can be repeated, each iteration with its own focal unit and scale. Starting from focus slot precision $p^{(0)}$, iteration ℓ reaches precision $p^{(\ell+1)} = \min\big((M+1)p^{(\ell)} - c,\ p+1\big) - 1$ with a $(p^{(\ell+1)}+1)$-bit focal unit u_ℓ, so that the fresh error matches the cleaning error. Thus each iteration multiplies the precision by about $a = M+1$ in D levels, where D is the depth of P. Since P is fixed, a and D are constants, and $T = O(\log p)$ iterations reach p bits. We write

$$\mathsf{Clean}^* := \mathsf{Clean}^{[u_{T-1}]} \circ \cdots \circ \mathsf{Clean}^{[u_0]}$$

for the iterated cleaning, and $u = u_{T-1}$ for its final focal unit.

Since $\log\|u_\ell\|_\infty^{\mathsf{can}} \approx p^{(\ell+1)}/k$, each level of iteration ℓ consumes $\log\Delta_\ell \approx p^{(\ell+1)}/k + \eta'$ bits. As in [CKSS25, Sec. 3.3], the precisions grow geometrically, so the total modulus consumption of cleaning is

$$\begin{aligned}\frac{D}{k}\big(p^{(0)}a + p^{(0)}a^2 + \cdots + p\big) + O(D\eta'\log p) &\approx \frac{p}{k}\cdot\frac{aD}{a-1} + O(D\eta'\log p) \\ &= O(p/k) + O(\log p).\end{aligned}$$

The precision-dependent part is $1/k$ of that of [CKSS25].

Remark 5. The constant can be improved by rescaling to the output scale $\Delta_{\ell+1}$ rather than Δ_ℓ, so that every fresh error in iteration ℓ is introduced at the output scale, as in the thrifty evaluation of h_1 [CKSS25, Sec. 4.1]. This reduces the factor $aD/(a-1)$ to about $a/(a-1)$, without changing the asymptotics; whether it applies depends on the circuit evaluating P.

Tree-like iterated cleaning. The iterated cleaning above uses focal units for the focus slots J_F throughout, so cleaning all slots requires $\mathsf{Clean}^* \circ \mathsf{Rot}_r$ for each of the k cosets, that is, kT cleanings. Alternatively, whenever a cleaning iteration doubles the precision target, we may halve the focus slots, so that $\log \|u\|_\infty^{\mathsf{can}} \approx p/k$, and hence the scale, stays about the same. Each ciphertext then splits into two, one for each half of its focus slots, which reduces the number of cleanings at the cost of more modulus. We describe it for h_1, which roughly doubles the precision in each iteration; for an order-M cleaning polynomial, the same construction splits into $M+1$ parts instead of two.

For $\ell \in [T+1]$ with $T = \log k$, let $J_\ell = 2^\ell[N/2^{\ell+1}]$, and define the precisions $p^{(\ell+1)} = 2p^{(\ell)} - c - 1$ from the starting precision $p^{(0)}$. Let $u_0 = 1$, and for $\ell \in [T]$, let $u_{\ell+1}$ be a $(p^{(\ell+1)}+1)$-bit focal unit for $J_{\ell+1}$ from the subring construction with $k = 2^{\ell+1}$. Algorithm 1 describes the tree-like cleaning.

Algorithm 1 Tree-like iterated cleaning with h_1

Require: ct with every slot within $2^{-p^{(0)}} \le \epsilon$ of $\mathcal{D}$
Ensure: $\{\mathsf{ct}_{T,r}\}_{r\in[k]}$, where $\mathsf{ct}_{T,r}$ holds the coset $r + J_F$ on J_F with precision about $k\,p^{(0)}$

$\mathsf{ct}_{0,0} \leftarrow \mathsf{ct}$
for $\ell \in [T]$ **do**
  for $r \in [2^\ell]$ **do**
    $\mathsf{ct}_{\ell+1,r} \leftarrow \mathsf{Clean}^{[u_{\ell+1}]}(\mathsf{ct}_{\ell,r})$
    $\mathsf{ct}_{\ell+1,r+2^\ell} \leftarrow \mathsf{Clean}^{[u_{\ell+1}]}\big(\mathsf{Rot}_{2^\ell}^{[u_\ell]}(\mathsf{ct}_{\ell,r})\big)$ ▷ moves $2^\ell + J_{\ell+1}$ to $J_{\ell+1}$
return $\{\mathsf{ct}_{T,r}\}_{r\in[k]}$

We show by induction on ℓ that $\mathsf{ct}_{\ell,r}$ holds the coset $r + J_\ell$ on J_ℓ with precision $p^{(\ell)}$, with every slot within ϵ of $\mathcal{D}$. For the inductive step, J_ℓ splits into the two halves $J_{\ell+1}$ and $2^\ell + J_{\ell+1}$: the first child keeps the first half $J_{\ell+1}$ in place, and the second child moves the second half onto $J_{\ell+1}$ by the rotation by 2^ℓ, which fixes u_ℓ and hence keeps the precision $p^{(\ell)}$. Therefore both inputs to the cleaning have precision $p^{(\ell)}$ on $J_{\ell+1}$, and by Proposition 16 with the focal unit $u_{\ell+1}$, both outputs reach precision $p^{(\ell+1)}$ there. Hence $\mathsf{ct}_{T,r}$ holds the same values as $\mathsf{Clean}^* \circ \mathsf{Rot}_r(\mathsf{ct})$ for each $r \in [k]$. It matches the target precision when $p' \approx k\,p^{(0)}$; otherwise, a few iterations of the previous method can be added before or after the tree.

The tree performs $2 + 4 + \cdots + k \approx 2k$ cleanings per digit instead of kT, reducing the number of relinearizations by a factor of about $T/2$, with the same

$k-1$ rotations. On the other hand, layer ℓ uses $\log \|u_{\ell+1}\|_\infty^{\mathsf{can}} \approx (2p^{(\ell)})/2^{\ell+1} \approx p^{(0)}$, so every layer works at a scale of about $p^{(0)} + \eta'$ bits, and the cleaning consumes about $DT(p^{(0)}+\eta') = O\big((p/k)\log k\big) + O(\log k)$ bits instead of $O(p/k) + O(\log p)$.

6.2 Cleaning-based Bootstrapping for Focused CKKS

Algorithm 2 describes the overall algorithm of the proposed bootstrapping.

Algorithm 2 Cleaning-based Focused Bootstrapping

Require: $\mathsf{ct}_{\mathsf{in}} \in \mathcal{R}_q^2$ encrypting $\Delta m + ue$ with $m = \sigma^{-1}(\iota_k(\boldsymbol{z}))$
Ensure: $\mathsf{ct}_{\mathsf{out}} \in \mathcal{R}_Q^2$ encrypting $\Delta m + ue_{\mathsf{out}}$

$(\hat{b}, \hat{a}) \leftarrow [\mathsf{ct}_{\mathsf{in}}]_q \in \mathcal{R}^2$ ▷ ModRaise: $\hat{b} + \hat{a}\cdot\mathsf{sk} = \Delta m + ue + qI$
$\mathsf{ct}_{\mathsf{msg}} \leftarrow \big(\lfloor \mathsf{mask}_F \hat{b} \rceil_u, \lfloor \mathsf{mask}_F \hat{a} \rceil_u\big) \bmod Q$ ▷ masked lift
$(\mathsf{ct}_{\mathsf{Re}}, \mathsf{ct}_{\mathsf{Im}}) \leftarrow \mathsf{C2S}\big((\hat{b}, \hat{a}) \bmod Q_{\mathsf{top}}\big)$ ▷ low precision
$\{\mathsf{ct}_{\mathsf{Re}}^{(i)}\}_{i\in[w]} \leftarrow \mathsf{DigitExtract}(\mathsf{ct}_{\mathsf{Re}})$
$\{\mathsf{ct}_{\mathsf{Im}}^{(i)}\}_{i\in[w]} \leftarrow \mathsf{DigitExtract}(\mathsf{ct}_{\mathsf{Im}})$
for $r \in [k]$ **do**
  for $i \in [w]$ **do**
    $\mathsf{ct}_{2r}^{(i)} \leftarrow \mathsf{Clean}^* \circ \mathsf{Rot}_r(\mathsf{ct}_{\mathsf{Re}}^{(i)})$ ▷ moves slots $r + J_F$ to J_F
    $\mathsf{ct}_{2r+1}^{(i)} \leftarrow \mathsf{Clean}^* \circ \mathsf{Rot}_r(\mathsf{ct}_{\mathsf{Im}}^{(i)})$
  $\mathsf{ct}_{2r} \leftarrow \sum_{i\in[w]} \beta^i \mathsf{ct}_{2r}^{(i)}$
  $\mathsf{ct}_{2r+1} \leftarrow \sum_{i\in[w]} \beta^i \mathsf{ct}_{2r+1}^{(i)}$
$\mathsf{ct}_{\mathsf{ovf}} \leftarrow \mathsf{S2C}_F \circ \mathsf{Recombine}(\{\mathsf{ct}_j\}_{j\in[2k]})$ ▷ partial S2C
return $\mathsf{ct}_{\mathsf{msg}} - \mathsf{ct}_{\mathsf{ovf}}$ ▷ recover focus slots

We describe its components as follows.

Masked lift. Let $(\hat{b}, \hat{a}) = [\mathsf{ct}_{\mathsf{in}}]_q \in \mathcal{R}^2$ be the centered lift of the input ciphertext. As in ModRaise, it satisfies $\hat{b} + \hat{a}\cdot\mathsf{sk} = \Delta m + ue + qI$ for some $I \in \mathcal{R}$.

Let $\mathsf{mask}_F = \sigma^{-1}(\iota_k(1,\ldots,1)) \in \mathcal{K}$ be the fractional mask of the focus slots. As in Section 5.6, we multiply the lift before rounding, now with focused rounding:

$$\lfloor \mathsf{mask}_F \hat{b} \rceil_u + \lfloor \mathsf{mask}_F \hat{a} \rceil_u \cdot \mathsf{sk} = \mathsf{mask}_F(\hat{b} + \hat{a}\cdot\mathsf{sk}) + u\, e_{\mathsf{rnd}},$$

where e_{rnd} has the form of a rescale error, so $\|e_{\mathsf{rnd}}\|_\infty^{\mathsf{can}} \le E_{\mathsf{RS}}$. Since m is supported on the focus slots, $\mathsf{mask}_F m = m$, and $\mathsf{mask}_F \cdot ue = u(\mathsf{mask}_F e)$ with $\|\mathsf{mask}_F e\|_\infty^{\mathsf{can}} \le \|e\|_\infty^{\mathsf{can}}$. Hence $\mathsf{ct}_{\mathsf{msg}}$ encrypts

$$\Delta m + q\,\mathsf{mask}_F I + u\big(\mathsf{mask}_F e + e_{\mathsf{rnd}}\big) \pmod{Q},$$

so the overflow now lies on the focus slots only. The masked lift consumes no modulus.

Coefficients to slots and digit extraction. The overflow branch starts from the unmasked lift, reduced modulo the top modulus $Q_{\mathsf{top}} \geq Q$. We use $\mathsf{C2S}$ and $\mathsf{DigitExtract}$ of [CKSS25, Sec. 3] verbatim, with low precision. $\mathsf{C2S}$ outputs $\mathsf{ct}_{\mathsf{Re}}$ and $\mathsf{ct}_{\mathsf{Im}}$, whose slots hold noisy coefficients of I in bit-reversed order. With the overflow bound $\|I\|_\infty < K$, as in [CKSS25] we may assume I is non-negative by adding K. $\mathsf{DigitExtract}$ maps the ciphertext with noisy integer slot values $I_j \in [0, \beta^w)$ to w ciphertexts $\mathsf{ct}^{(i)}$ whose slots lie within ϵ of the base-β digits of I_j, so that $I_j = \sum_{i\in[w]} \beta^i I_j^{(i)}$ with $I_j^{(i)} \in \mathcal{D} = \{0, \ldots, \beta-1\}$.

Cleaning all slots by rotation. For $r \in [k]$, the rotation Rot_r moves the coset $r + J_F$ to J_F. By Proposition 16 and the iterated cleaning, Clean^* brings the focus slots to p'-bit precision, with p' specified below, while every other slot stays within ϵ of $\mathcal{D}$. After the digit sums, ct_{2r} and ct_{2r+1} hold on J_F the values of the coset $r + J_F$ of the real and imaginary parts, respectively, with focused error. Alternatively, Algorithm 1 produces the cleaned digits for all $r \in [k]$ at once, with fewer cleanings.

Partial slots-to-coefficients. The overflow branch must compute $q\,\mathsf{mask}_F I$. Write $I(X) = \sum_{i\in[d]} I_i(X^d)\,X^i$ with $\deg I_i < 2k$. As $5^k \equiv 1 \pmod{4k}$, we have $(\zeta^{5^{k\ell}})^d = \zeta_{4k}$, so the focus slot $k\ell$ holds

$$I(\zeta^{5^{k\ell}}) = \sum_{i\in[d]} \zeta^{i5^{k\ell}}\, I_i(\zeta_{4k}), \qquad \ell \in [d].$$

That is, the values of I on J_F are $\mathcal{V}_F\,(I_i(\zeta_{4k}))_{i\in[d]}$, where $\mathcal{V}_F = (\zeta^{i5^{k\ell}})_{\ell,i\in[d]}$ consists of the first d columns of $\mathcal{V}_{N/2}$ at the rows of the focus slots. As $\mathsf{C2S}$ outputs in bit-reversed order, the focus slot $k\ell$ of ct_j holds the $\mathrm{rev}_{2k}(j)$-th coefficient of $I_{\mathrm{rev}_d(\ell)}$. Hence the focus slot $k\ell$ of

$$\mathsf{Recombine}(\{\mathsf{ct}_j\}_{j\in[2k]}) = \sum_{j\in[2k]} X^{d\,\mathrm{rev}_{2k}(j)} \cdot \mathsf{ct}_j$$

holds $I_{\mathrm{rev}_d(\ell)}(\zeta_{4k})$, since X^d acts as ζ_{4k} on the focus slots. Multiplication by a monomial is exact and preserves focused error. Then $\mathsf{S2C}_F = \mathsf{Linear}^{[u]}_{\mathcal{V}_F\mathcal{P}_{\mathrm{rev}}}$ of Lemma 7, with the diagonals scaled so that I is multiplied by q, gives $\mathsf{ct}_{\mathsf{ovf}}$ encrypting

$$q\,\mathsf{mask}_F I + u\,e' \pmod{Q}.$$

Its diagonals are supported on J_F, so the leftover background values of the cleaning are discarded. Subtracting it from $\mathsf{ct}_{\mathsf{msg}}$ leaves $\Delta m + u(\mathsf{mask}_F e + e_{\mathsf{rnd}} - e')$, as desired. As in [CKSS25, Thm. 3.4 and Sec. 5.2], this requires the cleaning to reach $p' = p + \log(q/\Delta) + \log N + \log K + O(1)$ bits for p-bit focus slot precision, accounting for the digit sum, the map from the coefficients to the focus slots, and the gap q/Δ, where $K = O(\sqrt{h})$ heuristically [CKSS25, Sec. 3.2].

6.3 Cost Analysis

Asymptotic modulus consumption. The base modulus q only needs the fractional part $(\Delta m + ue)/q$ to be small for DigitExtract. Since $\|\Delta m + ue\|_\infty^{\mathsf{can}} \leq \Delta + \|u\|_\infty^{\mathsf{can}} E = O(\Delta)$ bounds its coefficients, $\log q = \log \Delta + O(1) \approx p/k + O(1)$. Unlike Algorithm 2, the bootstrapping of the previous section keeps all p bits in the base modulus.

Above the base modulus, the consumption is as follows.

- The masked lift, Recombine, and the subtraction consume no modulus.
- C2S and DigitExtract run at low precision, independent of p.
- The iterated cleaning consumes $\frac{p'}{k} \cdot \frac{aD}{a-1} + O(D\eta' \log p')$ bits, by the analysis of the iterated cleaning.
- $\mathsf{S2C}_F$ consumes one level at scale about $p'/k + \eta'$ bits, or ℓ such levels with the ℓ-level butterfly decomposition.

In total, the precision-dependent modulus consumption is $O(p/k)$, and the remaining $O(\log p)$ term comes from the per-level overhead of the cleaning.

Operation count. Apart from the cleaning, the operations are those of [CKSS25]: C2S and DigitExtract are performed once for each of the real and imaginary parts, and $\mathsf{S2C}_F$ is a single linear transform on the d focus slots. The cleaning dominates the cost. [CKSS25] evaluates P on $2w$ digit ciphertexts for T iterations, that is, $2wT$ times. The iterated cleaning does so for each of the k cosets, $2wkT$ times, whereas the tree-like cleaning of Algorithm 1 evaluates P about $4wk$ times. Both additionally use $2w(k-1)$ rotations.

Concrete accounting. Table 2 estimates the modulus consumption for $p/k = 16$, so that the arithmetic scale stays fixed while the precision grows from 32 to 256 bits. We follow the setting of [CKSS25, Sec. 5]: $N = 2^{16}$, $h = 30$, $|I| \leq 16$, so $w = 5$ bits for each of the real and imaginary parts, and a per-level overhead of $\eta' = 12$ bits over the precision. A focus slot precision of $p^{(\ell)}$ bits uses the scale $\log \Delta_\ell = \log \|u\|_\infty^{\mathsf{can}} + \eta'$, where $\log \|u\|_\infty^{\mathsf{can}}$ exceeds $p^{(\ell)}/k$ by 0, 0.68, 1.10, and 1.73 bits for $k = 2, 4, 8, 16$, the largest excess in the corresponding column of Table 1. The cleaning starts at precision $p^{(0)}$, which requires a gap of $\log(q/\Delta) = p^{(0)} + 4$ [CKSS25, Sec. 5.2], and targets $p' = p + \log(q/\Delta) + \log N + w$, with $w = \log \beta^w \approx \log K + 1$. We use the thrifty evaluation of h_1, which maps precision $p^{(\ell)}$ to $p^{(\ell+1)} = 2p^{(\ell)} - 2$ and consumes $3 \log \Delta_{\mathsf{in}} - \log \Delta_{\mathsf{out}}$ bits per iteration [CKSS25, Sec. 4.1]. We estimate C2S and DigitExtract as 3 and 8 levels of $p^{(0)} + \eta'$ bits, and use two levels for S2C as in [CKSS25, Sec. 5]. For each k, we choose $p^{(0)}$ minimizing the total. For comparison, we apply the same estimation to [CKSS25] by taking $k = 1$, that is, without a focal unit and with the same thrifty cleaning. The estimation tends to underestimate the actual consumption of both methods. It ignores the rounding of scaling factors to the available primes, extra levels for constant multiplications and polynomial evaluation, and margins for the failure probability.

Table 2. Estimated modulus consumption in bits of Algorithm 2 and of [CKSS25], for equal precision p with $p/k = 16$. The total includes the base modulus $\log q$.

p	Method	$\log q$	C2S+DE	Cleaning	S2C	Total
32	Ours, $k = 2$	38	198	136	87	459
	[CKSS25]	54	198	175	150	577
64	Ours, $k = 4$	41	220	128	74	463
	[CKSS25]	88	220	203	218	729
128	Ours, $k = 8$	41	220	161	66	488
	[CKSS25]	152	220	359	346	1077
256	Ours, $k = 16$	42	220	196	64	522
	[CKSS25]	284	264	479	610	1637

In [CKSS25], the base modulus, the cleaning, and S2C all grow linearly with p. In ours, they shrink by the factor k, so the total stays nearly flat, and at $p = 256$ it is about 32% of that of [CKSS25]. The cost is the factor k in the slots and in the cleaning operations.

The starting precision $p^{(0)}$ trades the cleaning against C2S and DigitExtract. A higher $p^{(0)}$ saves cleaning iterations, but increases the base modulus and the cost of C2S and DigitExtract. For $k = 16$ and $p = 256$, the total is minimized at $p^{(0)} = 8$ with $T = 6$ iterations; raising $p^{(0)}$ to 32 saves two iterations but increases the total from 522 to 766 bits.

References

[AKP24] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using CKKS. Cryptology ePrint Archive, Paper 2024/1623, 2024.

[BCC+22] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. In *ACM SIGSAC CCS*, 2022.

[BCIV20] Carl Bootland, Wouter Castryck, Ilia Iliashenko, and Frederik Vercauteren. Efficiently processing complex-valued data in homomorphic encryption. *Journal of Mathematical Cryptology*, 14(1):55–65, 2020.

[BFG+26] Zvika Brakerski, Offir Friedman, Daniel Golan, Alon Gurny, Dolev Mutzari, and Ohad Sheinfeld. REFHE: Fully homomorphic ALU. In *Advances in Cryptology – EUROCRYPT*, 2026.

[BK25] Dan Boneh and Jaehyung Kim. Homomorphic encryption for large integers from nested residue number systems. In *Advances in Cryptology – CRYPTO*, 2025.

[BKSS24] Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping small integers with CKKS. In *Advances in Cryptology – ASIACRYPT*, 2024.

[CCKS23] Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In *ACM SIGSAC CCS*, pages 696–710, 2023.

[CCS19] Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT*, 2019.

[CDPR16] Ronald Cramer, Léo Ducas, Chris Peikert, and Oded Regev. Recovering short generators of principal ideals in cyclotomic rings. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 559–585. Springer, 2016.

[CHK+18] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT*, 2018.

[CHM+25] Hyunho Cha, Intak Hwang, Seonhong Min, Jinyeong Seo, and Yongsoo Song. MatriGear: Accelerating authenticated matrix triple generation with scalable prime fields via optimized HE packing. In *IEEE Symposium on Security and Privacy*, 2025.

[CIL21] Hao Chen, Ilia Iliashenko, and Kim Laine. When HEAAN meets FV: A new somewhat homomorphic encryption with reduced memory overhead. In *Cryptography and Coding – IMACC*, 2021.

[CKK20] Jung Hee Cheon, Dongwoo Kim, and Duhyeong Kim. Efficient homomorphic comparison methods with optimal complexity. In *Advances in Cryptology – ASIACRYPT*, pages 221–256, 2020.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *International conference on the theory and application of cryptology and information security*, pages 409–437. Springer, 2017.

[CKSS25] Hyeongmin Choe, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Leveraging discrete CKKS to bootstrap in high precision. In *ACM SIGSAC CCS*, 2025.

[CLPX18] Hao Chen, Kim Laine, Rachel Player, and Yuhou Xia. High-precision arithmetic in homomorphic encryption. In *Topics in Cryptology – CT-RSA*, 2018.

[CPL26] Gyeongwon Cha, Dongjin Park, and Joon-Woo Lee. Improved radix-based approximate homomorphic encryption for large integers via lightweight bootstrapped digit carry. In *Advances in Cryptology – EUROCRYPT*, 2026.

[DMPS24] Nir Drucker, Guy Moshkowich, Tomer Pelleg, and Hayim Shaul. BLEACH: Cleaning errors in discrete computations over CKKS. *Journal of Cryptology*, 37(1):3, 2024.

[Dum26] Jules Dumezy. Fully homomorphic encryption for small batches of complex numbers. Cryptology ePrint Archive, Paper 2026/2513, 2026.

[GV25] Robin Geelen and Frederik Vercauteren. Fully homomorphic encryption for cyclotomic prime moduli. In *Advances in Cryptology – EUROCRYPT*, 2025.

[GZ26] Mingyu Gao and Hongren Zheng. FHE for SIMD arithmetic logic units with amortized $O(1)$ bootstrapping per ciphertext. In *Advances in Cryptology – CRYPTO*, 2026.

[HHC19] Kyoohyung Han, Minki Hhan, and Jung Hee Cheon. Improved homomorphic discrete fourier transforms and FHE bootstrapping. *IEEE Access*, 2019.

[HK20] Kyoohyung Han and Dohyeong Ki. Better bootstrapping for approximate homomorphic encryption. In *Topics in Cryptology – CT-RSA*, 2020.

[HS14] Shai Halevi and Victor Shoup. Algorithms in HElib. In *Advances in Cryptology – CRYPTO*, Lecture Notes in Computer Science, 2014.

[HS18] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in HElib. In *Advances in Cryptology – CRYPTO*, 2018.

[Kim25] Jaehyung Kim. Efficient homomorphic integer computer from CKKS. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):873–898, 2025.

[Lem12] Franz Lemmermeyer. Ideal class groups of cyclotomic number fields I, 2012.

[LLK+22] Yongwoo Lee, Joon-Woo Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and HyungChul Kang. High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In *Advances in Cryptology – EUROCRYPT*, 2022.

[LLL+21] Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In *Advances in Cryptology – EUROCRYPT*, 2021.

[PZZ26] Chris Peikert, Doron Zarchy, and Guy Zyskind. High-precision exact FHE made simple, general, and fast. In *Advances in Cryptology – CRYPTO*, 2026.

[SSKM25] Hyewon Sung, Sieun Seo, Taekyung Kim, and Chohong Min. EvalRound+ bootstrapping and its rigorous analysis for CKKS scheme. *IEEE Access*, 2025.